Studio Nx

P1-07 가입 · 로그인 JS — 함수 설명 합본 (09~17)

2026-10-12

목차

1. ai-notes/javascript/P1-07 - 가입·로그인 JS/09_auth-ui_setFieldState.md
2. ai-notes/javascript/P1-07 - 가입·로그인 JS/10_auth-ui_initPasswordToggles.md
3. ai-notes/javascript/P1-07 - 가입·로그인 JS/11_auth-ui_initOAuthPlaceholders.md
4. ai-notes/javascript/P1-07 - 가입·로그인 JS/12_auth-ui_initAuthState.md
5. ai-notes/javascript/P1-07 - 가입·로그인 JS/13_auth-ui_initLoginForm.md
6. ai-notes/javascript/P1-07 - 가입·로그인 JS/14_home_initLoginModalFromHash.md
7. ai-notes/javascript/P1-07 - 가입·로그인 JS/15_join_initDuplicateChecks.md
8. ai-notes/javascript/P1-07 - 가입·로그인 JS/16_join_initPasswordRules.md
9. ai-notes/javascript/P1-07 - 가입·로그인 JS/17_join_initJoinForm.md

JS 09 — `setFieldState()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-101, FR-102)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	08 `logout()` (P1-02의 끝) · 10 `initPasswordToggles()`

1. 하는 일

입력칸 하나의 **상태**(초록, 빨강, 기본)와 그 아래의 **문구**를 바꿉니다.

호출	화면 (join.html의 이메일 칸)
`setFieldState(field, 'ok', '✓ Available')`	밑줄이 **초록**이 되고, 아래에 초록 글씨로 "✓ Available"이 나옵니다
`setFieldState(field, 'error', 'This email is already in use.')`	밑줄이 **빨강**이 되고, 아래에 빨강 글씨가 나옵니다. 화면 읽기 프로그램은 "잘못된 값"으로 읽습니다
`setFieldState(field, null)`	밑줄이 기본 색(보라)으로 돌아가고, 문구는 처음 HTML에 있던 안내 글로 돌아갑니다 (join.html은 빈 문자열)

`field`는 입력칸을 감싼 `.field` 요소입니다 (예: `#join-email-field`).

```
<div class="field" id="join-email-field">          ← is-ok / is-error 가 여기 붙음
  <input class="field__control" id="join-email" …>  ← aria-invalid="true" 가 여기 붙음
  <p class="field__msg" id="join-email-msg"></p>     ← 문구가 여기 들어감
</div>
```

2. 언제 쓰나

P1-07의 다른 함수들이 칸마다 결과를 보여 줄 때 이 함수를 부릅니다. 그래서 가장 먼저 만들었습니다.

부르는 곳 (P1-07)	칸	예
`initDuplicateChecks()`	이메일, 닉네임	Check를 누르면 ✓ Available 또는 This email is already in use.
`initPasswordRules()`	비밀번호, 비밀번호 확인	Use only English letters, numbers and symbols. / Passwords don't match.
`initJoinForm()`	서버가 422로 돌려준 칸	`details.nickname`이 있으면 닉네임 칸이 빨강이 됩니다

색은 `forms.css`가 정합니다. 이 함수는 **클래스와 속성만** 바꿉니다.

붙이는 것	`forms.css`가 하는 일
`.field.is-ok`	밑줄과 문구를 초록(`--c-ok`)으로 만듭니다
`.field.is-error` + `aria-invalid="true"`	밑줄과 문구를 빨강(`--c-error`)으로 만듭니다

3. 코드

import (파일 맨 위)

```
// (없음)
```

`auth-ui.js`에는 아직 import가 없습니다 (다음 함수들에서 `api.js`와 `auth.js`를 가져올 예정입니다). **이 함수는 import가 필요하지 않습니다.** `querySelector`, `classList`, `dataset`은 브라우저의 기본 기능입니다.

함수

```
export function setFieldState(field, state, message) {
  if (!field) return;
  const control = field.querySelector('.field__control');
  const msg = field.querySelector('.field__msg');

  field.classList.remove('is-ok', 'is-error');
  if (control) control.removeAttribute('aria-invalid');

  if (state === 'ok') {
    field.classList.add('is-ok');
  } else if (state === 'error') {
    field.classList.add('is-error');
    if (control) control.setAttribute('aria-invalid', 'true');
  }

  if (msg) {
    if (msg.dataset.hint === undefined) msg.dataset.hint = msg.textContent; // 처음 안내 글을 기억
    msg.textContent = message === undefined ? msg.dataset.hint : message;
  }
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위의 전통 문법입니다)

```
export function setFieldState(field, state, message) {
  if (!field) return;
  const control = field.querySelector('.field__control');
  const msg = field.querySelector('.field__msg');

  field.classList.toggle('is-ok', state === 'ok');
  field.classList.toggle('is-error', state === 'error');
  if (state === 'error') control?.setAttribute('aria-invalid', 'true');
  else control?.removeAttribute('aria-invalid');

  if (msg) {
    msg.dataset.hint ??= msg.textContent;
    msg.textContent = message ?? msg.dataset.hint;
  }
}
```

최신 문법	실제 코드 (전통 문법)	뜻
`control?.setAttribute(…)`	`if (control) control.setAttribute(…)`	`control`이 있을 때만 실행합니다

최신 문법	실제 코드 (전통 문법)	뜻
`msg.dataset.hint ??= msg.textContent`	`if (msg.dataset.hint === undefined) msg.dataset.hint = msg.textContent`	비어 있을 때만 넣습니다
`message ?? msg.dataset.hint`	`message === undefined ? msg.dataset.hint : message`	값이 없으면 대신 쓸 값입니다 (`??`는 `null`도 "없음"으로 봅니다)
`classList.toggle('is-ok', 조건)`	`remove`한 뒤 조건에 맞으면 `add`	조건이 참이면 붙이고, 거짓이면 뗍니다

문법 풀이

import 문법	뜻
`export function`	다른 파일(`join.js` 등)이 `import { setFieldState } from '../core/auth-ui.js'`로 가져다 씁니다

식	뜻
`if (!field) return;`	`field`가 없으면(`null` 등) 아무것도 하지 않고 끝냅니다
`field.querySelector('.field__control')`	`field` **안에서** 이 클래스를 가진 첫 요소를 찾습니다. 없으면 `null`입니다
`classList.remove('is-ok', 'is-error')`	두 클래스를 한 번에 뗍니다. 없어도 오류가 나지 않습니다
`setAttribute('aria-invalid', 'true')`, `removeAttribute(…)`	HTML 속성을 붙이거나 뗍니다
`msg.dataset.hint`	HTML의 `data-hint="…"` 속성입니다. 이 함수를 처음 부를 때 만들어서 안내 글을 보관합니다
`조건 ? A : B`	조건이 참이면 A, 아니면 B입니다 (오래된 전통 문법인 삼항 연산자)
`message === undefined`	세 번째 인자를 **넘기지 않았는지** 확인합니다. `''`(빈 문자열)를 넘기면 빈 문자열로 바꿉니다

4. 흐름

1. `field`가 없으면 끝냅니다.
2. 상태 표시(`is-ok`, `is-error`, `aria-invalid`)를 모두 지웁니다.
3. `'ok'`이면 초록, `'error'`이면 빨강과 `aria-invalid`를 붙입니다.
4. 문구를 바꿉니다. 넘긴 문구가 있으면 그것을, 없으면 처음의 안내 글을 씁니다.

5. 왜 이렇게

선택	이유
먼저 모두 지운 뒤 붙임	`ok`에서 `error`로 바뀔 때 `is-ok`가 남아서 초록과 빨강이 섞이는 일이 없습니다
색은 CSS에 두고, JS는 클래스만 바꿈	디자인을 바꿀 때 JS를 건드리지 않아도 됩니다. 같은 상태 규칙을 모든 폼(`forms.css`)이 씁니다
`aria-invalid="true"`	색을 볼 수 없는 사람도 화면 읽기 프로그램으로 "잘못된 입력"임을 알 수 있습니다. 문구 칸은 `aria-live="polite"`라서 바뀌면 읽어 줍니다
처음 안내 글을 `data-hint`에 보관	"2–20 characters" 같은 안내가 있는 칸에서 오류를 지우면 안내 글이 돌아옵니다. HTML을 고치지 않아도 됩니다

선택	이유
`field`, `control`, `msg`가 없어도 오류가 나지 않음	칸 모양이 조금 다른 폼(로그인 모달 등)에서도 안전하게 쓸 수 있습니다

6. 시험 (실제 join.html을 헤드리스 Chrome으로 열고 그 안에서 실행, 11개)

P1-02의 Node 시험과 달리, 이 함수는 실제 HTML과 CSS가 있어야 하므로 **브라우저 안에서** 시험했습니다 (Sentry를 끈 8001번 개발 서버).

시험	확인한 것
처음 상태	클래스가 없고, `aria-invalid`도 없으며, 문구는 빈 문자열입니다
ok, error, null, error→ok (4개)	클래스, `aria-invalid`, 문구가 표대로 바뀌고, 바뀔 때 이전 상태가 남지 않습니다
실제 밑줄 색	error는 `rgb(255, 138, 138)`, ok는 `rgb(127, 224, 168)`, 기본은 `rgba(163, 131, 255, 0.45)`입니다. `forms.css`가 실제로 적용되었습니다
안내 글이 있는 칸 (4개)	error 뒤에 null이면 "2–20 characters"로 복원되고, 문구 없이 ok여도 안내 글이 나오며, 빈 문자열을 넘기면 빈 문자열이 됩니다
`field`가 `null`	오류가 나지 않습니다

시험 중에 한 번 실패한 것: 처음에는 세 상태의 밑줄 색이 모두 같게 나왔습니다. `forms.css`의 색 전환 효과(`transition`) 때문에, 바꾼 직후에는 아직 이전 색이었습니다. 함수의 문제가 아니라 시험이 너무 빨리 읽은 것이었으므로, 0.6초 기다린 뒤 읽도록 고쳤습니다.

11개 모두 통과했습니다.

JS 10 — `initPasswordToggles()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-101, FR-103)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	09 `setFieldState()` · 11 `initOAuthPlaceholders()`

1. 하는 일

비밀번호 칸 오른쪽의 **Show / Hide** 버튼이 동작하게 합니다. 누를 때마다 입력한 비밀번호를 보였다가 가렸다가 합니다.

누르기 전	누른 뒤
`••••••••`, 버튼 **Show**	`Abcdef1!`, 버튼 **Hide**
`<input type="password">`, `aria-pressed="false"`	`<input type="text">`, `aria-pressed="true"`

입력한 값은 바뀌지 않고, 보이는 방식(`type`)만 바뀝니다.

2. 언제 쓰나

페이지가 열릴 때 한 번 부릅니다. 그러면 그 페이지의 Show 버튼이 모두 동작합니다.

페이지	버튼	부르는 곳
join.html	Password, Confirm password, 로그인 모달의 Password (3개)	`join.js`
home.html	로그인 모달의 Password	`home.js` (P1-07의 뒤 순서)
userInfo.html	현재, 새, 확인 비밀번호와 탈퇴 비밀번호 (4개)	`userinfo.js` (P1-10)

버튼과 입력칸은 HTML의 `aria-controls`로 짝지어져 있습니다.

```
<input class="field__control" id="join-password" type="password" …>
<button type="button" class="pw__toggle" aria-controls="join-password" aria-pressed="false">Show</button
>
```

3. 코드

import (파일 맨 위)

```
// (없음)
```

`auth-ui.js`에는 아직 import가 없습니다. **이 함수는 import가 필요하지 않습니다.** 브라우저의 기본 기능만 씁니다.

함수

```
export function initPasswordToggles(root = document) {
  const buttons = root.querySelectorAll('.pw__toggle');
  for (const button of buttons) {
    if (button.dataset.pwToggle === 'on') continue;
    button.dataset.pwToggle = 'on';
    button.addEventListener('click', () => {
      const input = document.getElementById(button.getAttribute('aria-controls'));
      if (!input) return;
      const show = input.type === 'password'; // 지금 가려져 있으면 → 보이게
      input.type = show ? 'text' : 'password';
      button.textContent = show ? 'Hide' : 'Show';
      button.setAttribute('aria-pressed', show ? 'true' : 'false');
    });
  }
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export function initPasswordToggles(root = document) {
  root.querySelectorAll('.pw__toggle:not([data-pw-toggle])').forEach((button) => {
    button.dataset.pwToggle = 'on';
    button.addEventListener('click', () => {
      const input = document.getElementById(button.getAttribute('aria-controls'));
      if (!input) return;
      const show = input.type === 'password';
      input.type = show ? 'text' : 'password';
      button.textContent = show ? 'Hide' : 'Show';
      button.ariaPressed = String(show);
    });
  });
}
```

최신 문법	실제 코드	뜻
`querySelectorAll('…:not([data-pw-toggle])')`	반복문 안에서 `if (… === 'on') continue;`	이미 연결한 버튼을 건너뜁니다
`.forEach((button) => { … })`	`for (const button of buttons) { … }`	하나씩 돌면서 처리합니다
`button.ariaPressed = String(show)`	`button.setAttribute('aria-pressed', show ? 'true' : 'false')`	속성을 바꿉니다 (`ariaPressed`는 2020년 무렵 생긴 속성 이름입니다)

문법 풀이

import 문법	뜻
`export function`	`join.js` 등이 가져다 씁니다

식	뜻
`root = document`	인자를 주지 않으면 페이지 전체에서 찾습니다 (기본값)
`root.querySelectorAll('.pw__toggle')`	범위 안의 Show 버튼을 **모두** 찾습니다
`for (const button of buttons)`	하나씩 꺼내서 반복합니다
`continue`	이번 차례를 건너뛰고 다음 버튼으로 갑니다
`button.dataset.pwToggle = 'on'`	HTML에 `data-pw-toggle="on"`을 붙여서 "연결함"이라고 표시합니다

식	뜻
`addEventListener('click', () => { … })`	누를 때마다 `{ … }`를 실행합니다
`getAttribute('aria-controls')`	버튼이 가리키는 입력칸의 `id`입니다
`show ? 'text' : 'password'`	보일 차례면 `'text'`, 아니면 `'password'`입니다

4. 흐름

1. 범위 안의 Show 버튼을 모두 찾습니다.
2. 이미 연결한 버튼은 건너뜁니다.
3. 각 버튼에 "누르면" 동작을 붙입니다. 누르면 짝이 되는 입력칸을 찾아서 `type`, 버튼 글자, `aria-pressed`를 함께 바꿉니다.

5. 왜 이렇게

선택	이유
같은 버튼에 두 번 연결하지 않음 (`data-pw-toggle`)	두 번 연결되면 한 번 누를 때 두 번 바뀌어서 **아무 일도 일어나지 않은 것처럼** 보입니다. 로그인 모달처럼 나중에 끼워 넣는 HTML이 있어서 함수가 여러 번 불릴 수 있습니다
`root` 인자	나중에 끼워 넣은 부분(모달 등)만 골라서 연결할 수 있습니다
입력칸은 `aria-controls`로 찾음	버튼 바로 앞의 칸을 찾는 방식보다, HTML 구조가 바뀌어도 안전합니다. 화면 읽기 프로그램도 같은 속성으로 짝을 압니다
`aria-pressed`도 함께 바꿈	화면 읽기 프로그램이 "Show, 눌림" 또는 "눌리지 않음"으로 상태를 읽어 줍니다
버튼 글자와 상태는 입력칸의 지금 `type`을 보고 정함	버튼 글자를 기준으로 하면, 다른 코드가 `type`을 바꿨을 때 어긋납니다

6. 시험 (실제 join.html을 헤드리스 Chrome으로 열고 그 안에서 실행, 9개)

`join.js`가 페이지를 열 때 이미 이 함수를 부르므로, 시험에서 **두 번 더** 불러서 중복 연결이 막히는지도 함께 확인했습니다.

시험	확인한 것
연결	페이지의 Show 버튼 3개(가입 2개, 로그인 모달 1개)가 모두 연결되었습니다
누르기	처음에는 `password · Show · false`이고, 한 번 누르면 `text · Hide · true`가 됩니다 (세 번 불렀어도 **한 번만** 바뀝니다). 다시 누르면 처음으로 돌아갑니다
값	입력한 `Abcdef1!`은 그대로입니다
짝	확인 칸의 버튼은 확인 칸만 바꿉니다
`root`	새로 넣은 버튼은 연결 전에는 동작하지 않고, `initPasswordToggles(box)` 뒤에는 동작합니다
짝 칸이 없는 버튼	눌러도 오류가 나지 않고, 글자도 그대로입니다

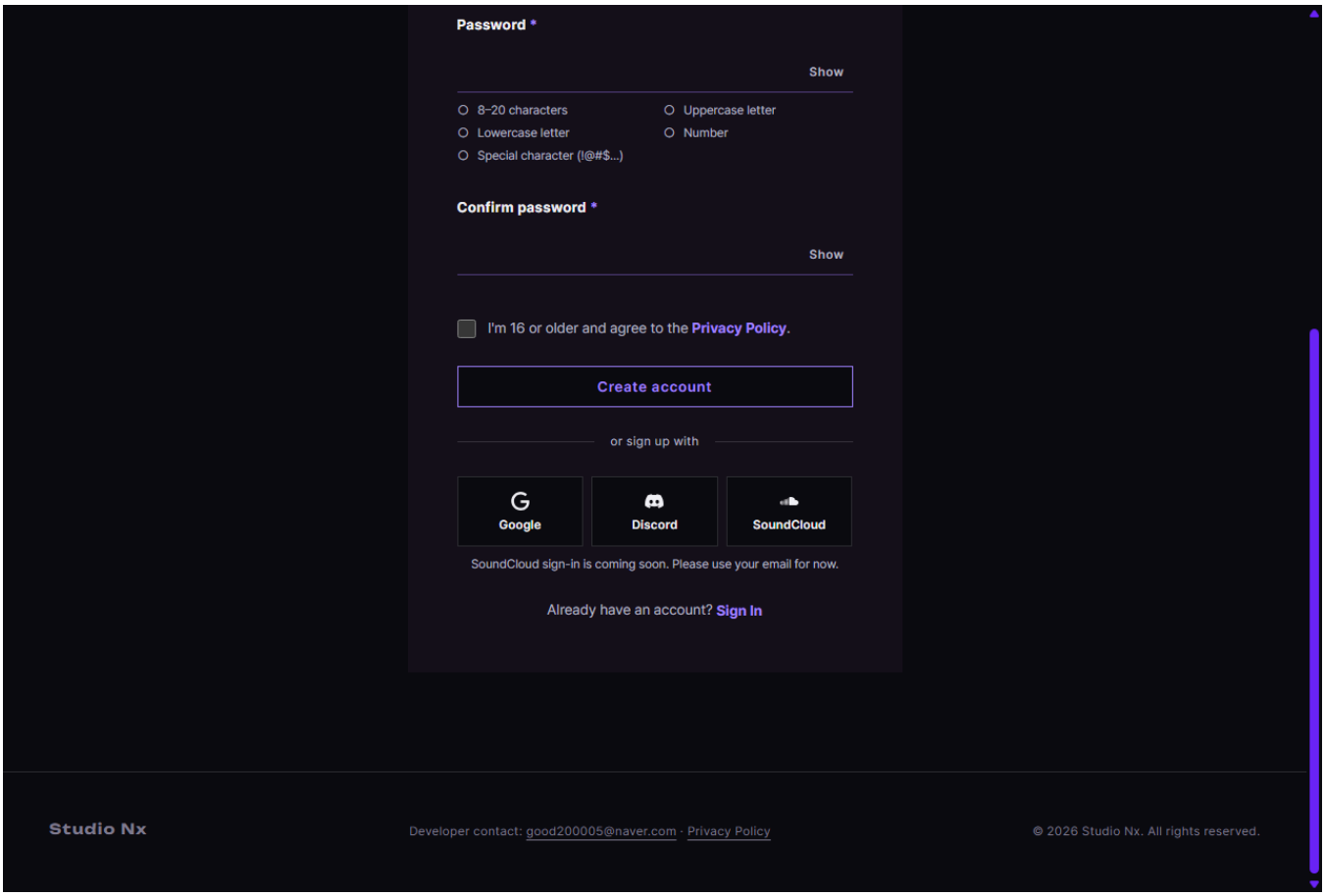
9개 모두 통과했습니다.

JS 11 — `initOAuthPlaceholders()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`, `studionx/frontend/static/css/auth.css` (한 줄)
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-110)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	10 `initPasswordToggles()` · 12 `initAuthState()`

1. 하는 일

소셜 로그인 버튼(**Google, Discord, SoundCloud**)을 누르면, 버튼 줄 바로 아래에 "아직 준비 중"이라는 안내 한 줄을 보여 줍니다. 연동은 나중에 할 예정이라(FR-110) 지금은 자리만 있습니다.



누른 버튼	버튼 아래 안내
Google	Google sign-in is coming soon. Please use your email for now.
Discord	Discord sign-in is coming soon. Please use your email for now.
SoundCloud	SoundCloud sign-in is coming soon. Please use your email for now.

다른 버튼을 누르면 **같은 줄의 글만 바뀝니다** (줄이 늘어나지 않습니다).

2. 언제 쓰나

페이지가 열릴 때 한 번 부릅니다.

페이지	버튼 줄
join.html	가입 폼 아래의 "or sign up with" 줄과 로그인 모달 안의 줄 (버튼 6개)
home.html	로그인 모달 안의 줄 (`home.js`, P1-07의 뒤 순서)

나중에 연동할 때: 이 함수 안의 "안내 쓰기" 부분을 `location.href = '/api/v1/auth/oauth/<provider>/start'`로 바꾸면 됩니다 (템플릿 주석에 적혀 있으며, PRD 01 개정이 필요합니다).

3. 코드

import (파일 맨 위)

```
// (없음)
```

이 함수는 import가 필요하지 않습니다. 브라우저의 기본 기능만 씁니다.

함수

```
/** 소셜 로그인 서비스 이름 (안내 문구용) */
const OAUTH_NAMES = { google: 'Google', discord: 'Discord', soundcloud: 'SoundCloud' };

export function initOAuthPlaceholders(root = document) {
  const buttons = root.querySelectorAll('.oauth[data-provider]');
  for (const button of buttons) {
    if (button.dataset.oauthReady === 'on') continue;
    button.dataset.oauthReady = 'on';
    button.addEventListener('click', () => {
      const provider = button.dataset.provider;
      const name = OAUTH_NAMES[provider] || provider;
      const row = button.closest('.oauth-row') || button.parentElement;
      let note = row.nextElementSibling;
      if (!note || !note.classList.contains('oauth-note')) {
        note = document.createElement('p');
        note.className = 'field__msg oauth-note';
        note.setAttribute('role', 'status');
        row.after(note);
      }
      note.textContent = `${name} sign-in is coming soon. Please use your email for now.`;
    });
  }
}
```

CSS 한 줄 추가 (`auth.css`)

```
.oauth-note { margin-top: var(--sp-2); text-align: center; }
```

글자 크기와 색은 기존의 `.field__msg`(작은 회색 글씨)를 함께 써서 따로 정하지 않았습니다.

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export function initOAuthPlaceholders(root = document) {
  root.querySelectorAll('.oauth[data-provider]:not([data-oauth-ready])').forEach((button) => {
    button.dataset.oauthReady = 'on';
    button.addEventListener('click', () => {
      const { provider } = button.dataset;
      const name = OAUTH_NAMES[provider] ?? provider;
      const row = button.closest('.oauth-row') ?? button.parentElement;
      let note = row.nextElementSibling;
      if (!note?.classList.contains('oauth-note')) {
        note = Object.assign(document.createElement('p'), { className: 'field__msg oauth-note', role: 's
tatus' });
        row.after(note);
      }
      note.textContent = `${name} sign-in is coming soon. Please use your email for now.`;
    });
  });
}
```

최신 문법	실제 코드	뜻
`const { provider } = button.dataset`	`const provider = button.dataset.provider`	구조 분해입니다. 칸을 같은 이름의 변수로 꺼냅니다
`OAUTH_NAMES[provider] ?? provider`	`OAUTH_NAMES[provider] || provider`	없으면 대신 쓸 값입니다 (여기서는 이름이 빈 문자열일 일이 없으므로 둘의 결과가 같습니다)
`!note?.classList.contains(…)`	`!note || !note.classList.contains(…)`	`note`가 없거나, 안내 줄이 아니면 참입니다
`Object.assign(요소, { … })`	`className = …`과 `setAttribute('role', …)`를 한 줄씩 씀	여러 속성을 한 번에 넣습니다

문법 풀이

import 문법	뜻
`export function`	`join.js`와 `home.js`가 가져다 씁니다 (`OAUTH_NAMES`는 `export`가 없어서 이 파일 안에서만 씁니다)

식	뜻
`'.oauth[data-provider]'`	`oauth` 클래스이면서 `data-provider` 속성이 있는 요소입니다
`button.dataset.provider`	HTML의 `data-provider="google"`을 읽으면 `'google'`입니다
`OAUTH_NAMES[provider] || provider`	표에 있으면 보기 좋은 이름(`SoundCloud`)을, 없으면 받은 문자열을 그대로 씁니다
`button.closest('.oauth-row')`	버튼에서 **위로** 올라가며 가장 가까운 버튼 줄을 찾습니다
`row.nextElementSibling`	버튼 줄 **바로 다음** 요소입니다. 안내 줄이 이미 있으면 그것입니다
`document.createElement('p')`, `row.after(note)`	새 `<p>`를 만들어 버튼 줄 바로 뒤에 넣습니다
`` `${name} sign-in …` ``	템플릿 문자열입니다. `${ }` 안의 값을 문자열에 끼워 넣습니다 (2015년에 나온 문법이며, 지금은 기본입니다)

4. 흐름

1. 범위 안의 소셜 버튼을 모두 찾습니다 (이미 연결한 버튼은 건너뜁니다).

2. 버튼을 누르면 서비스 이름을 정하고, 버튼 줄 바로 아래에 안내 줄이 없으면 만듭니다.
3. 안내 줄의 글을 그 서비스의 문구로 바꿉니다.

5. 왜 이렇게

선택	이유
`alert` 대신 화면 안에 안내 한 줄 (템플릿은 둘 다 허용)	`alert`는 화면을 멈추고 확인 버튼을 눌러야 합니다. 안내 한 줄은 하던 일을 방해하지 않습니다
안내 줄을 하나만 두고 글만 바꿈	여러 번 눌러도 줄이 쌓이지 않습니다
`role="status"`	화면 읽기 프로그램이 새 안내를 방해 없이(`polite`) 읽어 줍니다
"Please use your email for now."	막혔다는 말만 하지 않고, 지금 할 수 있는 일을 알려 줍니다
같은 버튼에 두 번 연결하지 않음 (`data-oauth-ready`)	10번과 같은 이유입니다
이름 표(`OAUTH_NAMES`)	`soundcloud`를 `SoundCloud`처럼 대소문자를 바르게 보여 줍니다. 표에 없는 서비스가 추가되어도 오류 없이 그 이름으로 보입니다
CSS는 기존 `.field__msg`를 함께 씀	다른 칸 아래 문구와 같은 크기와 색이 됩니다. 추가한 것은 여백과 가운데 정렬 한 줄뿐입니다

6. 시험 (실제 join.html을 헤드리스 Chrome으로 열고 그 안에서 실행, 8개)

시험	확인한 것
연결	버튼 6개(가입 3개, 로그인 모달 3개)가 모두 연결되었고, 한 번 더 불러도 중복되지 않습니다
누르기 전	안내 줄이 없습니다
Google	줄 바로 아래에 `<p class="field__msg oauth-note" role="status">`와 Google 문구가 나옵니다
Discord	안내 줄은 그대로 1개이고, 글만 Discord로 바뀝니다
SoundCloud	"SoundCloud"로 표기됩니다
페이지 이동	없습니다 (`/join.html` 그대로)
모양	가운데 정렬, 위 여백 8px입니다
표에 없는 서비스	`github`를 그대로 표시하고, 오류가 나지 않습니다

8개 모두 통과했습니다. 위의 캡처는 같은 방식으로 SoundCloud를 누른 뒤에 찍었습니다.

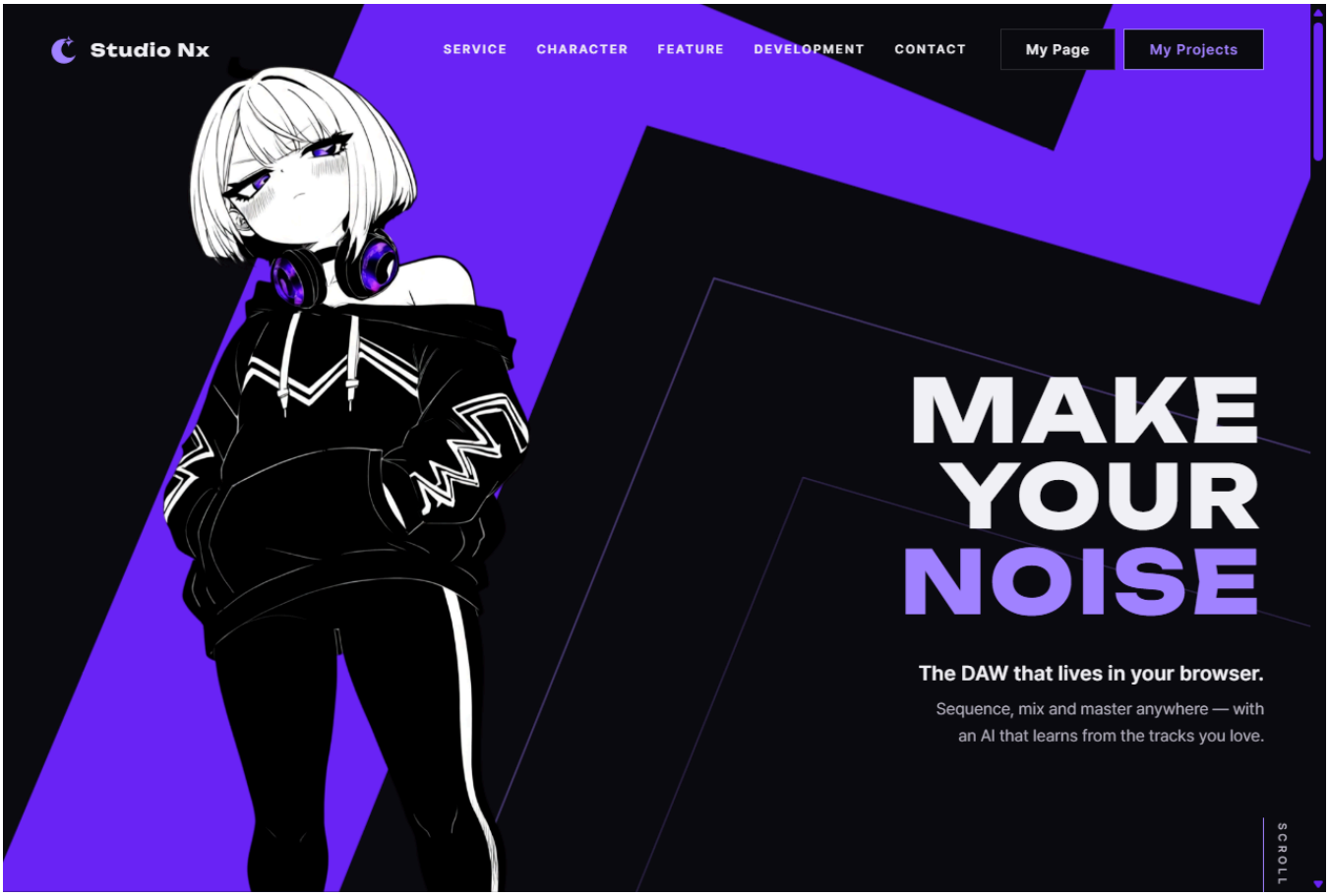
JS 12 — `initAuthState()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-105, FR-109)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	11 `initOAuthPlaceholders()` · 13 `initLoginForm()`

1. 하는 일

페이지를 열 때 **로그인 상태를 확인**해서 `<html>`에 표시합니다. 그러면 `auth.css`가 헤더 오른쪽 버튼을 바꿉니다.

상태	`<html data-auth>`	헤더 오른쪽 버튼
확인 중 (서버 응답을 기다림, 보통 0.1초 안)	`"pending"`	**비어 있습니다** (자리만 차지합니다)
로그인하지 않음	속성 없음	**Sign In, Join Us**
로그인함	`"in"`	**My Page, My Projects**



돌려주는 값은 로그인 상태면 `true`, 아니면 `false`입니다 (`Promise`).

2. 언제 쓰나

모든 페이지가 열릴 때 한 번 부릅니다 (이미 각 페이지 스크립트에 들어 있습니다).

페이지	부르는 곳
home.html	`home.js`
myPage.html, myProject.html, userInfo.html	`mypage.js`, `myproject.js`, `userinfo.js`
privacy.html	`privacy.js`

자동 로그인과의 관계 (FR-109): "Keep me signed in"으로 로그인했다면 브라우저를 닫았다 열어도 리프레시 쿠키(3일)가 남아 있습니다. 그래서 이 함수가 재발급에 성공하고, 헤더에 My Page와 My Projects가 보입니다.

페이지를 열 때 일어나는 일

순서	일어나는 일
①	`data-auth="pending"`이 되어 헤더 버튼이 잠깐 보이지 않습니다
②	`isLoggedIn()`이 `POST /api/v1/auth/refresh`(백엔드 33)를 보냅니다
③-a	200(쿠키 있음)이면 `data-auth="in"`이 되어 My Page와 My Projects가 보입니다. 받은 토큰은 메모리에 남아서 다음 API 요청에 씁니다
③-b	401(쿠키 없음)이면 속성을 지우고 Sign In과 Join Us가 보입니다

3. 코드

import (파일 맨 위)

```
import { isLoggedIn } from './auth.js';
```

`auth-ui.js`의 첫 import이며 **이번에 추가했습니다**. 이 함수가 P1-02의 06번 `isLoggedIn()`을 씁니다.

함수

```
export async function initAuthState() {
  const root = document.documentElement;
  root.dataset.auth = 'pending';
  const loggedIn = await isLoggedIn();
  if (loggedIn) {
    root.dataset.auth = 'in';
  } else {
    root.removeAttribute('data-auth');
  }
  return loggedIn;
}
```

템플릿에는 `export function`으로 되어 있었는데, 서버 응답을 기다려야 해서 **`async`로 바꿨습니다**. 부르는 쪽(`initAuthState();`)은 그대로 두어도 됩니다. 기다리지 않고 다음 줄로 넘어가고, 응답이 오면 헤더가 바뀝니다.

CSS (이미 있음, `auth.css`)

```
:root:not([data-auth="in"]) [data-auth-show="in"],
:root[data-auth="in"] [data-auth-show="out"] { display: none; }
:root[data-auth="pending"] [data-auth-show] { visibility: hidden; }
```

```
<!-- header.html -->
<button … data-auth-show="out">Sign In</button>   <a … data-auth-show="out">Join Us</a>
<a href="myPage.html" data-auth-show="in">My Page</a>   <a href="myProject.html" data-auth-show="in">My Projects</a>
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export async function initAuthState() {
  const root = document.documentElement;
  root.dataset.auth = 'pending';
  const loggedIn = await isLoggedIn();
  root.toggleAttribute('data-auth', loggedIn);
  if (loggedIn) root.dataset.auth = 'in';
  return loggedIn;
}
```

최신 문법	실제 코드	뜻
`root.toggleAttribute('data-auth', loggedIn)`	`if … else root.removeAttribute('data-auth')`	조건이 거짓이면 속성을 지웁니다 (참이면 빈 속성을 붙이고, 다음 줄이 `'in'`으로 덮어씁니다)

이 함수는 원래 짧아서 최신 문법과의 차이가 작습니다. 실제 코드의 `if … else`가 더 읽기 쉽습니다.

문법 풀이

import 문법	뜻
`import { isLoggedIn } from './auth.js'`	같은 폴더의 `auth.js`가 `export`한 함수를 가져옵니다

식	뜻
`document.documentElement`	페이지의 `<html>` 요소입니다
`root.dataset.auth = 'pending'`	`<html data-auth="pending">`으로 만듭니다
`async function`, `await isLoggedIn()`	서버 응답이 올 때까지 이 함수 안에서만 기다립니다 (페이지의 다른 동작은 멈추지 않습니다). Promise 설명
`root.removeAttribute('data-auth')`	속성 자체를 지웁니다. 그러면 CSS의 "로그인하지 않음" 규칙이 적용됩니다

4. 흐름

1. `<html data-auth="pending">`으로 헤더 버튼을 잠깐 숨깁니다.
2. 로그인 상태를 확인합니다 (메모리에 토큰이 없으면 재발급을 시도합니다).
3. 로그인 상태면 `"in"`으로 바꾸고, 아니면 속성을 지웁니다.
4. `true` 또는 `false`를 돌려줍니다.

5. 왜 이렇게

선택	이유
먼저 `pending`으로 숨김	바로 "Sign In, Join Us"를 보여 줬다가 0.1초 뒤 "My Page"로 바뀌면 **깜빡입니다**. 그래서 확인하는 동안은 버튼 자리만 비워 둡니다 (`visibility: hidden`은 자리를 그대로 두므로 헤더가 흔들리지 않습니다)
로그아웃 상태면 속성을 **지움** (`'out'`으로 바꾸지 않음)	CSS는 "속성이 없으면 로그아웃"으로 짜여 있습니다. 그래서 JS가 꺼져 있거나 오류가 나도 Sign In과 Join Us는 보입니다
판단은 06번 `isLoggedIn()`에 맡김	토큰, 쿠키, 재발급 규칙은 P1-02와 P1-05에 있습니다. 이 함수는 결과를 화면에 옮기기만 합니다
`async`로 만들고 결과를 돌려줌 (템플릿은 일반 함수)	마이페이지처럼 "로그인되어 있어야 다음 일을 하는" 페이지는 `if (await initAuthState())`로 기다릴 수 있습니다. 헤더만 바꾸는 페이지는 기다리지 않아도 됩니다

참고: 로그인하지 않은 방문자는 페이지를 열 때마다 `/auth/refresh`에서 401을 받습니다. 정상적인 흐름이지만, 개발자 도구의 네트워크 탭에는 빨간 401이 한 줄씩 보입니다 (백엔드 33의 "쿠키 없음").

6. 시험 (실제 home.html을 헤드리스 Chrome으로 열고 그 안에서 실행, 9개)

실제 서버(Sentry를 끈 8001번)에서 **가입과 로그인을 실제로 한 뒤** 확인했습니다. 시험 계정은 끝난 뒤 DB에서 지웠습니다.

시험	확인한 것
페이지를 연 뒤 (로그인하지 않음)	`data-auth`가 없고, 헤더에 보이는 버튼은 Sign In과 Join Us입니다
부르자마자	`data-auth="pending"`이고, 헤더 버튼이 하나도 보이지 않습니다
응답을 받은 뒤	`false`이고, 속성이 없습니다
가입 → 로그인 → 메모리 토큰 지움 → 다시 부름	`true`, `data-auth="in"`이고, 헤더에는 My Page와 My Projects가 보이며, 재발급으로 메모리 토큰도 생깁니다
로그아웃 → 다시 부름	`false`이고, 속성이 없으며, 헤더에는 Sign In과 Join Us가 보입니다

"보이는 버튼"은 실제로 계산된 스타일(`display`, `visibility`)로 셌습니다. 즉 `auth.css`가 실제로 적용되는지까지 확인했습니다. 9개 모두 통과했습니다. 위의 캡처는 시험이 끝날 때 로그인 상태로 찍었습니다.

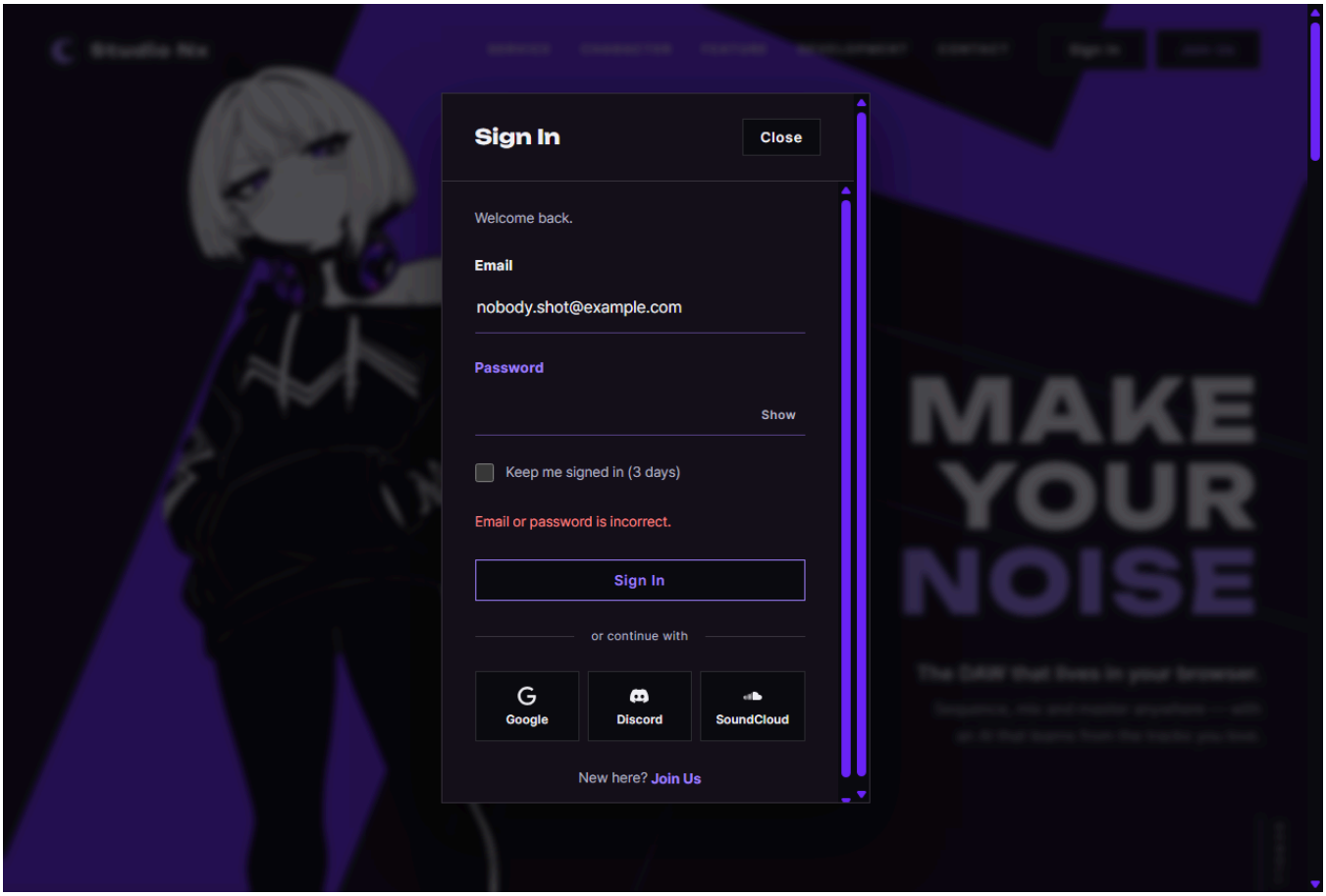
JS 13 — `initLoginForm()`

항목	내용
파일	`studionx/frontend/static/js/core/auth-ui.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-103, FR-109)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	12 `initAuthState()` · 14 `initLoginModalFromHash()`

1. 하는 일

로그인 모달의 **Sign In** 버튼을 누르면 서버에 로그인하고, 결과에 따라 화면을 바꿉니다. **이 함수 덕분에 사람이 화면에서 직접 로그인할 수 있게 되었습니다.**

경우	화면
맞음, 주소에 `next` 없음 (home.html에서 Sign In)	모달이 닫히고 헤더가 **My Page, My Projects**로 바뀝니다
맞음, 주소에 `next` 있음 (`home.html#login?next=%2FmyPage.html`)	그 페이지(`/myPage.html`)로 이동합니다
틀림, 또는 없는 이메일	모달 안에 빨간 글씨로 "Email or password is incorrect."가 나오고, **비밀번호 칸만 비운 뒤** 그 칸에 커서를 둡니다
정지된 계정 (비밀번호는 맞음)	빨간 글씨로 "This account is disabled."가 나옵니다 (서버 문구)
빈 칸	브라우저가 "이 입력란을 작성하세요"라고 알리고, 서버로 보내지 않습니다



2. 언제 쓰나

로그인 모달이 있는 페이지가 열릴 때 한 번 부릅니다.

페이지	부르는 곳	성공하면
home.html	`home.js`의 `initLoginForm({ onSuccess: null })`	기본 동작을 합니다 (위의 표)
join.html	`join.js` (같은 방식)	기본 동작을 합니다. 17번 `initJoinForm`을 만들 때 `onSuccess`로 "홈으로 이동"을 넣을 예정입니다

누르면 일어나는 일

순서	일어나는 일	관련
①	빈 칸과 이메일 모양을 **브라우저가** 먼저 검사합니다 (HTML의 `required`, `type="email"`)	
②	Sign In 버튼을 비활성화하고(두 번 누르기 방지), 오류 글을 숨깁니다	
③	`POST /api/v1/auth/login {email, password, remember}`를 보냅니다	JS 02, 백엔드 32
④-성공	`setAccessToken(access_token)`으로 메모리에 저장합니다. 리프레시 쿠키는 서버가 심습니다	JS 01
⑤-성공	주소의 `next`를 확인해서 이동하거나, 모달을 닫고 헤더를 바꿉니다	JS 05 `parseLoginHash`
④-실패	오류 글을 보여 주고, 비밀번호 칸을 비운 뒤 커서를 둡니다	
⑥	버튼을 다시 켭니다	

3. 코드

import (파일 맨 위)

```
import { api, setAccessToken } from './api.js';
import { isLoggedIn, parseLoginHash } from './auth.js';
```

`api`, `setAccessToken`, `parseLoginHash`를 이번에 추가했습니다. 이 함수가 셋 다 씁니다 (`isLoggedIn`은 12번이 씁니다).

함수

```
export function initLoginForm(options = {}) {
  const form = document.getElementById('login-form');
  if (!form || form.dataset.loginReady === 'on') return;
  form.dataset.loginReady = 'on';

  const email = document.getElementById('login-email');
  const password = document.getElementById('login-password');
  const remember = document.getElementById('login-remember');
  const error = document.getElementById('login-error');
  const submit = form.querySelector('button[type="submit"]');

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    error.hidden = true;
    submit.disabled = true;
    try {
      const data = await api.post(
        '/auth/login',
        { email: email.value, password: password.value, remember: remember.checked },
        { auth: false },
      );
      setAccessToken(data.access_token);
      const next = parseLoginHash().next;
      if (options.onSuccess) {
        options.onSuccess(next);
      } else if (next) {
        location.href = next;
      } else {
        const modal = document.getElementById('login-modal');
        if (modal && modal.open) modal.close();
        document.documentElement.dataset.auth = 'in';
        form.reset();
      }
    } catch (err) {
      if (err.status === 401 || err.status === 422) {
        error.textContent = 'Email or password is incorrect.';
      } else {
        error.textContent = err.message || 'Something went wrong. Please try again.';
      }
      error.hidden = false;
      password.value = '';
      password.focus();
    } finally {
```

```
      submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export function initLoginForm({ onSuccess } = {}) {
  const form = document.getElementById('login-form');
  if (!form || form.dataset.loginReady) return;
  form.dataset.loginReady = 'on';
  const [email, password, remember, error] =
    ['login-email', 'login-password', 'login-remember', 'login-error'].map((id) => document.getElementBy
Id(id));
  const submit = form.querySelector('[type="submit"]');

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    error.hidden = true;
    submit.disabled = true;
    try {
      const { access_token } = await api.post('/auth/login',
        { email: email.value, password: password.value, remember: remember.checked }, { auth: false });
      setAccessToken(access_token);
      const { next } = parseLoginHash();
      if (onSuccess) return onSuccess(next);
      if (next) return location.assign(next);
      document.getElementById('login-modal')?.close();
      document.documentElement.dataset.auth = 'in';
      form.reset();
    } catch (err) {
      error.textContent = [401, 422].includes(err.status)
        ? 'Email or password is incorrect.'
        : err.message ?? 'Something went wrong. Please try again.';
      error.hidden = false;
      password.value = '';
      password.focus();
    } finally {
      submit.disabled = false;
    }
  });
}
```

최신 문법	실제 코드	뜻
`function f({ onSuccess } = {})`	`function f(options = {}) + options.onSuccess`	인자 객체에서 바로 꺼냅니다 (구조 분해)
`const [a, b] = [...].map(…)`	한 줄에 하나씩 `const a = …`	여러 변수를 한 번에 만듭니다
`const { access_token } = await …`	`const data = await …` + `data.access_token`	응답에서 칸을 꺼냅니다
`modal?.close()`	`if (modal && modal.open) modal.close()`	있을 때만 실행합니다

최신 문법	실제 코드	뜻
`[401, 422].includes(err.status)`	`err.status === 401 || err.status === 422`	둘 중 하나인지 확인합니다
`err.message ?? '…'`	`err.message || '…'`	없으면 대신 쓸 글입니다 (빈 문자열일 때는 결과가 다릅니다. 실제 코드는 빈 문자열일 때도 대신 쓸 글을 씁니다)

문법 풀이

import 문법	뜻
`import { api, setAccessToken } from './api.js'`	같은 폴더의 `api.js`에서 두 이름을 가져옵니다

식	뜻
`form.addEventListener('submit', …)`	폼을 보낼 때(버튼 클릭, Enter) 실행합니다. 브라우저 검사(`required`)를 **통과해야만** 이 일이 일어납니다
`event.preventDefault()`	브라우저가 페이지를 새로 여는 원래의 전송을 막습니다. 대신 JS가 보냅니다
`try { … } catch (err) { … } finally { … }`	실패하면 `catch`를, 성공이든 실패든 마지막에 `finally`를 실행합니다
`err.status`	`ApiError`의 HTTP 상태 번호입니다
`{ auth: false }`	로그인 요청에는 토큰을 붙이지도 않고, 재발급하지도 않습니다
`modal.open`, `modal.close()`	`<dialog>`가 열려 있는지 확인하고, 닫습니다
`form.reset()`	폼의 입력을 처음 상태로 되돌립니다
`location.href = next`	그 주소로 이동합니다

4. 흐름

1. 폼이 없거나 이미 연결했으면 끝냅니다.
2. 폼을 보낼 때 오류 글을 숨기고 버튼을 끕니다.
3. 로그인을 요청합니다. 성공하면 토큰을 저장하고, `onSuccess`를 부르거나 `next`로 이동하거나, 모달을 닫고 헤더를 바꿉니다.
4. 실패하면 오류 글을 보여 주고, 비밀번호를 비우고, 커서를 둡니다.
5. 버튼을 다시 켭니다.

5. 왜 이렇게

선택	이유
401과 422는 같은 문구로 보여 줌	서버(29번)와 같은 원칙으로, 무엇이 틀렸는지 알려 주지 않습니다. 422(형식 틀림)도 화면에서는 같은 문구입니다
403과 그 밖의 오류는 서버 문구를 그대로 보여 줌	정지된 계정("This account is disabled.")은 비밀번호가 맞은 본인에게만 오는 응답이라서 그대로 보여 줍니다. 네트워크 오류도 `api.js`의 문구로 보여 줍니다
실패하면 **비밀번호만** 비움	대부분 비밀번호를 잘못 친 경우입니다. 이메일은 다시 치지 않아도 되게 남겨 둡니다
전송 중에는 버튼을 끔	두 번 눌러서 요청이 두 번 가는 것을 막습니다
`onSuccess`가 없으면 기본 동작 (템플릿은 페이지마다 넘기라고 되어 있었음)	home.html의 `onSuccess`가 `null`이어도 바로 쓸 수 있습니다. 다른 동작이 필요한 페이지(join.html)만 넘기면 됩니다

선택	이유
`next`는 `parseLoginHash()`로 얻음	JS 05가 `safeNextPath`로 검사한 값이라서, 바깥 사이트로 보내지는 일이 없습니다
헤더는 다시 확인하지 않고 `data-auth="in"`으로 바꿈	방금 로그인에 성공했으므로 `/auth/refresh`를 다시 부를 필요가 없습니다

6. 시험 (실제 home.html에서 실제 서버로, 12개)

시험 계정은 실제 가입 API로 만들었고, 끝난 뒤 DB에서 지웠습니다.

시험	확인한 것
연결	`home.js`가 페이지를 열 때 이미 연결했습니다
틀린 비밀번호	전송 중에 버튼이 꺼지고, "Email or password is incorrect."가 나오며, 이메일은 남고 비밀번호 칸은 비워지며 커서가 비밀번호 칸에 있습니다. 버튼은 다시 켜지고, 토큰은 없으며, 모달은 열려 있습니다
없는 이메일	같은 문구가 나옵니다
빈 칸	`/auth/login` 요청이 하나도 나가지 않습니다 (브라우저 검사)
성공 (`next` 없음)	오류 글이 숨겨지고, 모달이 닫히며, `data-auth="in"`이 되고, 폼이 비워집니다. 메모리 토큰이 있고, 재발급도 200입니다 (쿠키도 생겼습니다)
성공 (`#login?next=%2FmyPage.html%3Ftab%3Dusage`)	`/myPage.html?tab=usage`로 이동하려고 합니다 (시험에서는 브라우저의 Navigation API로 이동만 막고 목적지를 읽었습니다)
`onSuccess`를 준 경우	`onSuccess('/myPage.html?tab=usage')`로 불립니다
두 번 부름	중복으로 연결되지 않습니다

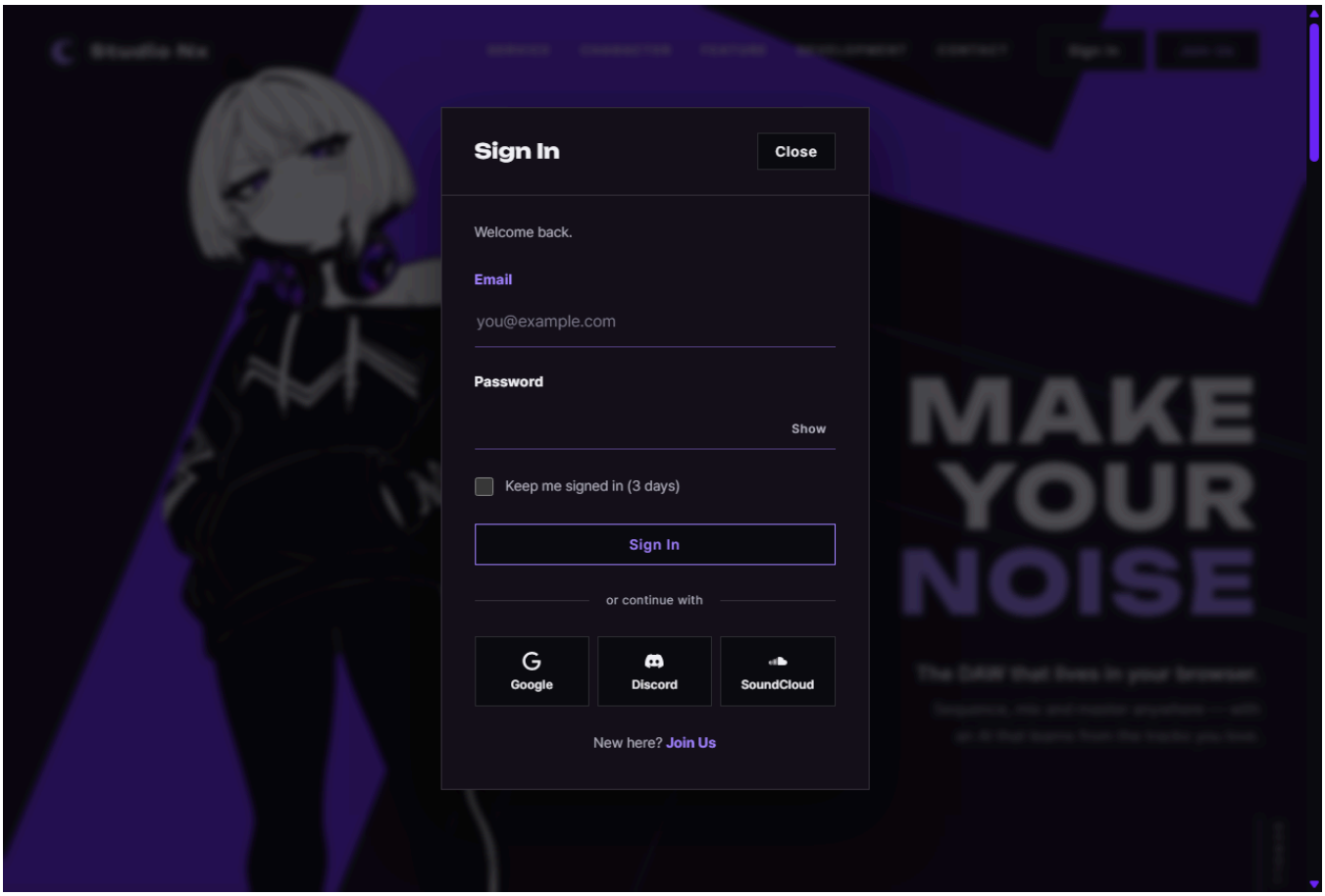
12개 모두 통과했습니다. 위의 캡처는 없는 이메일과 틀린 비밀번호로 보낸 뒤에 찍었습니다.

JS 14 — `initLoginModalFromHash()`

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-103: 로그인이 필요한 페이지 → 로그인 → 원래 페이지)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	13 `initLoginForm()` · 15 `initDuplicateChecks()`

1. 하는 일

주소가 `home.html#login…`이면 **로그인 모달을 자동으로 엽니다.** 모달을 닫으면 주소에서 `#login…`을 지웁니다.



들어온 주소	결과
`home.html`	모달이 닫혀 있습니다 (평소대로)
`home.html#login`	모달이 열립니다
`home.html#login?next=%2FmyProject.html%3Fsort%3Dtitle%23new`	모달이 열리고, 로그인하면 `/myProject.html?sort=title#new`로 갑니다 (13)
`home.html#features`	모달이 닫혀 있고, 그 구역으로 스크롤만 합니다

2. 언제 쓰나

로그인이 필요한 페이지가 로그인하지 않은 사람을 홈으로 보낼 때 이 주소를 씁니다. 이 함수는 **P1-02, P1-05, P1-07이 이어지는 흐름의 가운데 고리**입니다.

순서	화면	일어나는 일	관련
①	myProject.html (로그인하지 않음)	`requireLogin()` 이 재발급에서 401을 받고 `home.html#login?next=…` 로 이동합니다	JS 07, JS 04
②	home.html	**이 함수**가 모달을 자동으로 엽니다	
③	로그인 모달	로그인에 성공하면 주소의 `next` 로 이동합니다	JS 13, JS 05
④	myProject.html	`requireLogin()` 이 재발급에서 200을 받고 그대로 진행합니다	

로그인하지 않고 **Close**를 누르면 주소가 `home.html` 로 바뀝니다. 그래서 새로 고침해도 모달이 다시 열리지 않습니다.

3. 코드

import (`home.js` 맨 위)

```
import { initAuthState, initLoginForm, initPasswordToggles, initOAuthPlaceholders } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { parseLoginHash } from '../core/auth.js';
```

`parseLoginHash` 줄을 이번에 추가했습니다. 이 함수는 그것만 씁니다.

함수

```
export function initLoginModalFromHash() {
  const modal = document.getElementById('login-modal');
  if (!modal) return;

  const openIfLoginHash = () => {
    if (parseLoginHash().open && !modal.open) modal.showModal();
  };

  modal.addEventListener('close', () => {
    if (location.hash.startsWith('#login')) {
      history.replaceState(null, '', location.pathname + location.search);
    }
  });
  window.addEventListener('hashchange', openIfLoginHash);
  openIfLoginHash();
}
```

`home.js` 아래쪽에서 이미 `includePartials()`(모달 HTML 끼우기) 다음에 이 함수를 부르고 있습니다. 모달이 있어야 찾을 수 있으므로 순서가 중요합니다.

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export function initLoginModalFromHash() {
  const modal = document.getElementById('login-modal');
  if (!modal) return;
  const openIfLoginHash = () => parseLoginHash().open && !modal.open && modal.showModal();
  modal.addEventListener('close', () => {
    if (location.hash.startsWith('#login')) history.replaceState(null, '', location.pathname + location.
search);
  });
  addEventListener('hashchange', openIfLoginHash);
  openIfLoginHash();
}
```

최신 문법	실제 코드	뜻
`조건 && 조건 && 실행()`	`if (조건 && 조건) 실행()`	`&&`를 "참이면 다음을 실행"하는 데 쓰는 짧은 습관입니다. 읽기 어려워서 실제 코드는 `if`를 씁니다
`addEventListener(…)` (앞에 `window.` 가 없음)	`window.addEventListener(…)`	전역 함수는 `window.`를 생략할 수 있습니다

이 함수에는 최신 문법만의 기능이 거의 없습니다. 차이는 "짧게 쓰는 습관" 정도입니다.

문법 풀이

import 문법	뜻
`import { parseLoginHash } from '../core/auth.js'`	`../`는 한 폴더 위(`static/js/`)라는 뜻이므로, `static/js/core/auth.js`를 가리킵니다

식	뜻
`const openIfLoginHash = () => { … }`	이름을 붙인 화살표 함수입니다. 아래에서 두 번(처음, `hashchange`) 씁니다
`modal.showModal()`, `modal.open`	`<dialog>`를 모달로 열고, 열려 있는지 확인합니다
`modal.addEventListener('close', …)`	모달이 **어떤 이유로든** 닫힐 때 실행합니다 (Close 버튼, Esc, 로그인 성공 후의 `close()`)
`location.hash.startsWith('#login')`	주소의 `#` 부분이 `#login`으로 시작하는지 확인합니다
`history.replaceState(null, '', 새 주소)`	페이지를 다시 열지 않고 **주소 글자만** 바꿉니다. 방문 기록도 새로 쌓지 않습니다
`window.addEventListener('hashchange', …)`	페이지 안에서 `#…` 부분만 바뀔 때 실행합니다

4. 흐름

1. 모달이 없으면 끝냅니다.
2. 모달이 닫힐 때 주소에서 `#login…`을 지우도록 연결합니다.
3. 주소의 `#` 부분이 바뀔 때 다시 확인하도록 연결합니다.
4. 지금 주소를 확인해서 `#login`이면 모달을 엽니다.

5. 왜 이렇게

선택	이유
`next`를 따로 저장하지 않음 (템플릿에는 "저장해 두었다가"라고 되어 있었음)	모달이 열려 있는 동안 주소에 `#login?next=…`가 그대로 있습니다. 13번이 로그인에 성공한 순간 `parseLoginHash()`로 직접 읽으므로, 저장 장소를 하나 더 둘 필요가 없습니다
닫으면 `#login…`을 지움	그대로 두면 새로 고침할 때마다 모달이 다시 뜹니다
`replaceState`를 씀 (`location.hash = ''`가 아님)	`hash`를 바꾸면 방문 기록이 한 칸 쌓이고, 주소 끝에 `#`가 남습니다. `replaceState`는 둘 다 생기지 않습니다
`hashchange`도 받음 (템플릿에는 없었음)	이미 home.html을 보고 있을 때 `#login` 링크를 누르거나 주소창에 입력해도 모달이 열립니다
`!modal.open`을 확인	이미 열린 모달에 `showModal()`을 또 부르면 브라우저 오류가 납니다
바깥 주소인 `next`는 걸러짐	판단은 JS 05를 거쳐 JS 03 `safeNextPath`가 합니다. 모달은 열리지만 `next`는 `null`이므로, 로그인하면 홈에 머뭅니다

6. 시험 (실제 home.html을 헤드리스 Chrome으로 열고 그 안에서 실행, 8개)

시험	확인한 것
`home.html#login?next=%2FmyPage.html`로 열기	모달이 열리고, `parseLoginHash()`는 `{open: true, next: "/myPage.html"}`입니다
닫기	주소가 `/home.html`이 됩니다 (해시 없음)
페이지 안에서 `#login`으로 이동	모달이 열리고(`hashchange`), 닫으면 다시 지워집니다
`#features`	모달이 열리지 않습니다
`#login?next=https%3A%2F%2Fevil.com`	모달은 열리고, `next`는 `null`입니다
`home.html`(해시 없이)로 열기	모달이 닫혀 있습니다

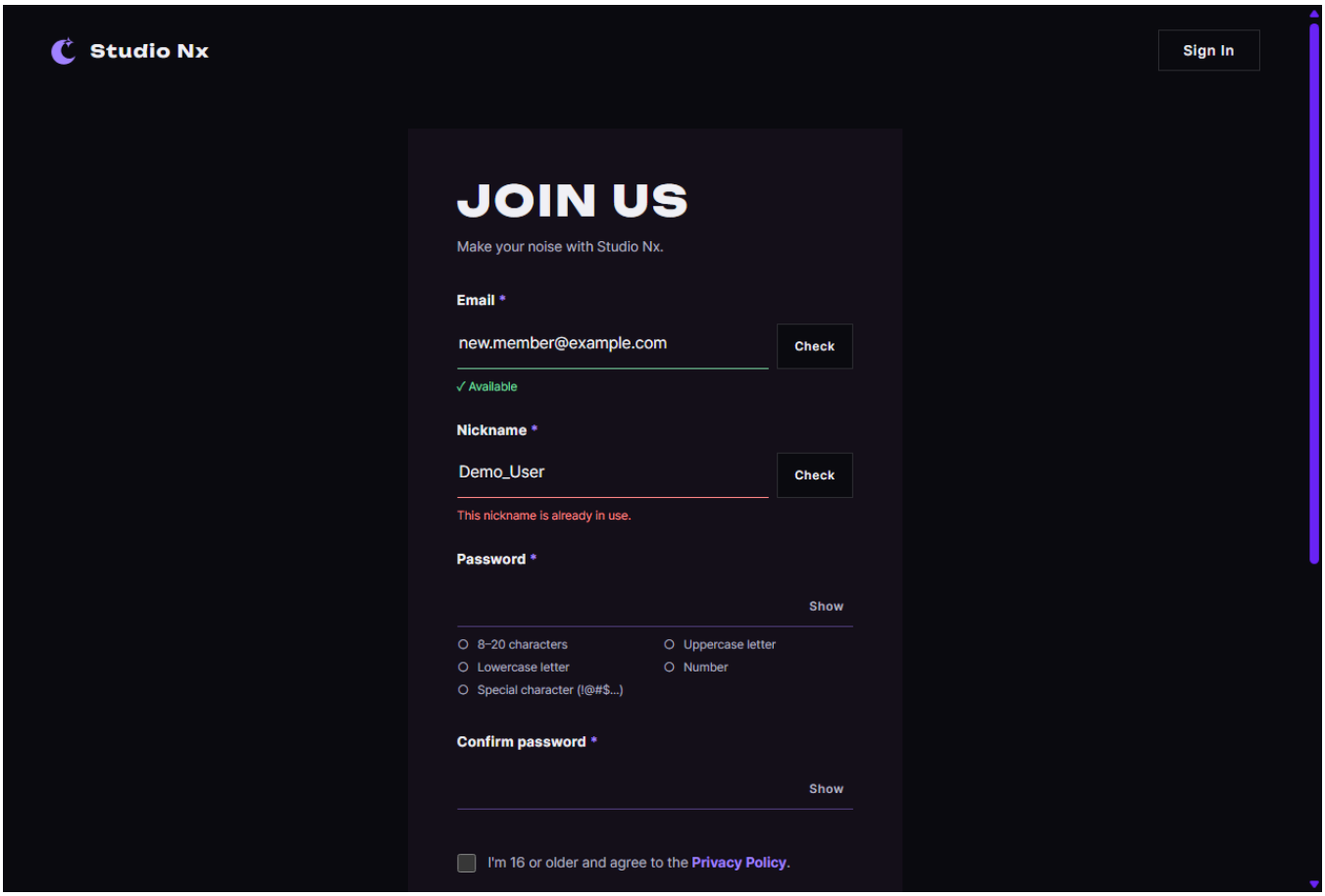
8개 모두 통과했습니다. 위의 캡처는 `home.html#login?next=%2FmyProject.html%3Fsort%3Dtitle%23new`로 열고, 첫 접속 로딩 화면이 끝난 뒤에 찍었습니다.

JS 15 — `initDuplicateChecks()`

항목	내용
파일	`studionx/frontend/static/js/pages/join.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-101 실시간 중복 확인, FR-108 탈퇴 후 3일)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	14 `initLoginModalFromHash()` · 16 `initPasswordRules()`

1. 하는 일

join.html의 이메일 칸과 닉네임 칸에 입력한 값이 **이미 쓰이고 있는지** 서버에 물어서 칸 아래에 보여 줍니다. **Check** 버튼을 누르거나, 입력을 멈추고 0.5초가 지나면 묻습니다.



입력	칸 아래
(빈 칸), `abc`, `kim@naver`	빨강 "Enter a valid email address."이고, **서버에 묻지 않습니다**
없는 이메일	잠깐 "Checking..."이 나온 뒤 초록 "✓ Available"이 나옵니다
가입된 이메일 (`DEMO@Example.com`처럼 대소문자가 달라도)	빨강 "This email is already in use."
탈퇴한 지 3일이 안 된 이메일	빨강 "You can sign up again after Oct 9." (날짜는 **사용자의 시간대** 기준)
닉네임 1자, 21자	빨강 "Use 2–20 characters."이고, 서버에 묻지 않습니다

입력	칸 아래
가입된 닉네임 (`Demo_User` ↔ `demo_user`)	빨강 "This nickname is already in use."

입력을 고치면 결과가 바로 지워집니다. 다시 확인해야 가입할 수 있습니다.

2. 언제 쓰나

join.html이 열릴 때 `join.js`가 한 번 부릅니다. 묻는 곳은 P1-04에서 만든 API입니다.

칸	API	서버
이메일	`GET /api/v1/auth/check-email?email=`	백엔드 23
닉네임	`GET /api/v1/auth/check-nickname?nickname=`	백엔드 24

확인 결과는 가입 전송(17번 `initJoinForm`)이 씁니다. "✓ Available"이 나온 값은 칸에 `data-checked="그 값"`으로 남습니다. 가입 버튼을 누를 때 지금 값과 같은지 보고, 다르면(확인하지 않았거나 고친 뒤라면) 가입을 막습니다.

3. 코드

import (`join.js` 맨 위)

```
import { initLoginForm, initPasswordToggles, initOAuthPlaceholders, setFieldState } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { api } from '../core/api.js';
```

`api` 줄을 이번에 추가했습니다. 이 함수는 `api`(서버에 묻기)와 09번 `setFieldState`(칸 상태)를 씁니다.

함수

```
/** 중복 확인 칸마다 다른 것 — 주소 · 문구 · 서버에 묻기 전 형식 검사 */
const CHECKS = {
  email: {
    path: '/auth/check-email?email=',
    taken: 'This email is already in use.',
    invalid: 'Enter a valid email address.',
    isValid: (input) => input.value.trim() !== '' && input.checkValidity(),
  },
  nickname: {
    path: '/auth/check-nickname?nickname=',
    taken: 'This nickname is already in use.',
    invalid: 'Use 2–20 characters.',
    isValid: (input) => input.value.trim().length >= 2 && input.value.trim().length <= 20,
  },
};

export function initDuplicateChecks() {
  for (const kind of Object.keys(CHECKS)) {
    const rule = CHECKS[kind];
    const field = document.getElementById(`join-${kind}-field`);
    const input = document.getElementById(`join-${kind}`);
```

```
    const button = document.querySelector(`[data-check="${kind}"]`);
    if (!field || !input) continue;
    let timer = null;

    const check = async () => {
      clearTimeout(timer);
      delete field.dataset.checked; // 이번 확인을 통과해야 다시 남김
      const value = input.value.trim();
      if (!rule.isValid(input)) {
        setFieldState(field, 'error', rule.invalid);
        return;
      }
      setFieldState(field, null, 'Checking…');
      let result;
      try {
        result = await api.get(rule.path + encodeURIComponent(value), { auth: false });
      } catch (err) {
        if (input.value.trim() !== value) return; // 그사이 입력이 바뀜 — 옛 응답은 버림
        setFieldState(field, 'error', err.status === 422 ? rule.invalid : err.message);
        return;
      }
      if (input.value.trim() !== value) return;
      if (result.available) {
        field.dataset.checked = value;
        setFieldState(field, 'ok', '✓ Available');
      } else if (result.reason === 'REJOIN_TOO_SOON') {
        const day = new Date(result.available_at).toLocaleDateString('en-US', { month: 'short', day: 'nu
meric' });
        setFieldState(field, 'error', `You can sign up again after ${day}.`);
      } else {
        setFieldState(field, 'error', rule.taken);
      }
    };

    input.addEventListener('input', () => {
      delete field.dataset.checked; // 값이 바뀌면 다시 확인해야 함
      setFieldState(field, null);
      clearTimeout(timer);
      if (rule.isValid(input)) timer = setTimeout(check, 500);
    });
    if (button) button.addEventListener('click', check);
  }
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다. 바뀌는 부분만 적었습니다)

```
for (const [kind, rule] of Object.entries(CHECKS)) { … }
…
const { available, reason, available_at } = await api.get(…);
…
button?.addEventListener('click', check);
```

최신 문법	실제 코드	뜻
`for (const [kind, rule] of Object.entries(CHECKS))`	`for (const kind of Object.keys(CHECKS))` + `const rule = CHECKS[kind]`	열쇠와 값을 한 번에 꺼냅니다

최신 문법	실제 코드	뜻
`const { available, reason, available_at } = await …`	`result = await …` + `result.available`	응답에서 칸을 꺼냅니다 (구조 분해)
`button?.addEventListener(…)`	`if (button) button.addEventListener(…)`	버튼이 있을 때만 실행합니다

문법 풀이

import 문법	뜻
`import { api } from '../core/api.js'`	`pages/`에서 한 폴더 위로 올라가 `core/api.js`를 가리킵니다

식	뜻
`` `join-${kind}-field` ``	`kind`가 `'email'`이면 `'join-email-field'`가 됩니다
`input.checkValidity()`	브라우저의 기본 검사(`type="email"`, `required`, `maxlength`)를 통과하면 `true`입니다
`setTimeout(check, 500)`, `clearTimeout(timer)`	0.5초 뒤에 실행하도록 예약하고, 예약을 취소합니다. 입력할 때마다 취소하고 다시 예약하므로, **입력을 멈춘 뒤** 0.5초가 지나야 실행됩니다
`encodeURIComponent(value)`	주소에 넣을 수 있게 바꿉니다 (`+` → `%2B`, `@` → `%40`). `a+b@x.com`의 `+`가 공백으로 바뀌는 것을 막습니다
`delete field.dataset.checked`	`data-checked` 속성을 지웁니다
`new Date(…).toLocaleDateString('en-US', { month: 'short', day: 'numeric' })`	UTC 시각을 **이 컴퓨터의 시간대**로 바꿔서 `Oct 9` 모양으로 만듭니다

4. 흐름

Check를 누를 때 (또는 자동 확인)

1. 예약된 자동 확인을 취소하고, 지난번 통과 표시를 지웁니다.
2. 형식이 틀리면 빨간 문구를 보여 주고 끝냅니다 (서버에 묻지 않습니다).
3. "Checking…"을 보여 주고 서버에 묻습니다.
4. 응답이 오기 전에 입력이 바뀌었으면 응답을 **버립니다**.
5. 사용할 수 있으면 초록 문구와 `data-checked`를, 탈퇴한 지 3일이 안 되었으면 날짜 문구를, 사용 중이면 빨간 문구를 보여 줍니다.

입력할 때

1. 지난 결과를 지웁니다.
2. 형식이 맞으면 0.5초 뒤에 자동 확인을 하도록 예약합니다.

5. 왜 이렇게

선택	이유
서버에 묻기 전에 형식을 검사	`abc`처럼 틀린 값은 서버까지 보낼 필요가 없습니다. 서버도 422로 막지만(23, 24), 요청을 아낄 수 있습니다
자동 확인은 형식이 맞을 때만	`kim@na`까지 쳤을 때 빨간 "Enter a valid…"가 뜨면 입력 중인 사람을 재촉하는 셈입니다. 그래서 다 쳤을 때만 묻습니다. Check 버튼을 누르면 형식 문구도 보여 줍니다

선택	이유
0.5초 기다림 (템플릿 권장)	글자마다 묻지 않습니다. `kim@naver.com` 13자를 칠 때 요청이 13번에서 1번으로 줄어듭니다
늦게 온 응답은 버림	`demo@…`를 확인하는 중에 다른 값으로 고치면, 늦게 온 "사용 중" 결과가 새 값에 붙을 수 있습니다. 그래서 응답이 왔을 때 칸의 값이 물어본 값과 같은지 확인합니다
확인할 때마다 `data-checked`를 먼저 지움	시험에서 찾은 빈틈입니다. 통과한 뒤 (입력 이벤트 없이) 값이 바뀌면, "사용 중"이 나와도 옛 통과 표시가 남아 있었습니다. 이제는 이번 확인을 통과해야만 표시가 남습니다
날짜는 사용자의 시간대로	서버는 UTC(`…Z`)로 줍니다. 한국 사용자에게는 한국 날짜로 보여야 합니다
칸마다 다른 부분은 `CHECKS` 표로 정리	이메일과 닉네임은 동작이 같고, 주소, 문구, 형식 규칙만 다릅니다. 같은 코드를 두 번 쓰지 않기 위해서입니다

6. 시험 (실제 join.html에서 실제 서버로, 15개)

데모 계정(`demo@example.com`, `demo_user`)과 시험용으로 **탈퇴한 지 1일 된 계정**을 DB에 넣고 확인했습니다 (탈퇴 계정은 끝난 뒤 지웠습니다).

시험	확인한 것
빈 칸, `abc` + Check	"Enter a valid email address."가 나오고, 서버 요청은 0번입니다
없는 이메일	누르자마자 "Checking…"이 나온 뒤 "✓ Available"이 나오고, `data-checked`가 남습니다
`DEMO@Example.com`	"This email is already in use."가 나오고, `data-checked`가 없습니다
탈퇴한 지 1일 된 이메일	"You can sign up again after Oct 9."
입력	결과가 바로 지워지고, 0.3초에는 아직 묻지 않으며, 0.5초가 지나면 자동으로 확인해서 ✓가 나옵니다
`auto.ch`처럼 덜 친 입력	자동으로 확인하지 않습니다 (빨간 글 없음, 요청 0번)
확인 중에 값을 바꿈	옛 값의 "사용 중" 결과가 새 값에 붙지 않습니다
닉네임 `a`, `DEMO_USER`, `" fresh_nick "`, 탈퇴한 지 1일 된 닉네임	차례대로 2–20자 문구, 사용 중, ✓(`data-checked="fresh_nick"`), 재가입 날짜 문구가 나옵니다

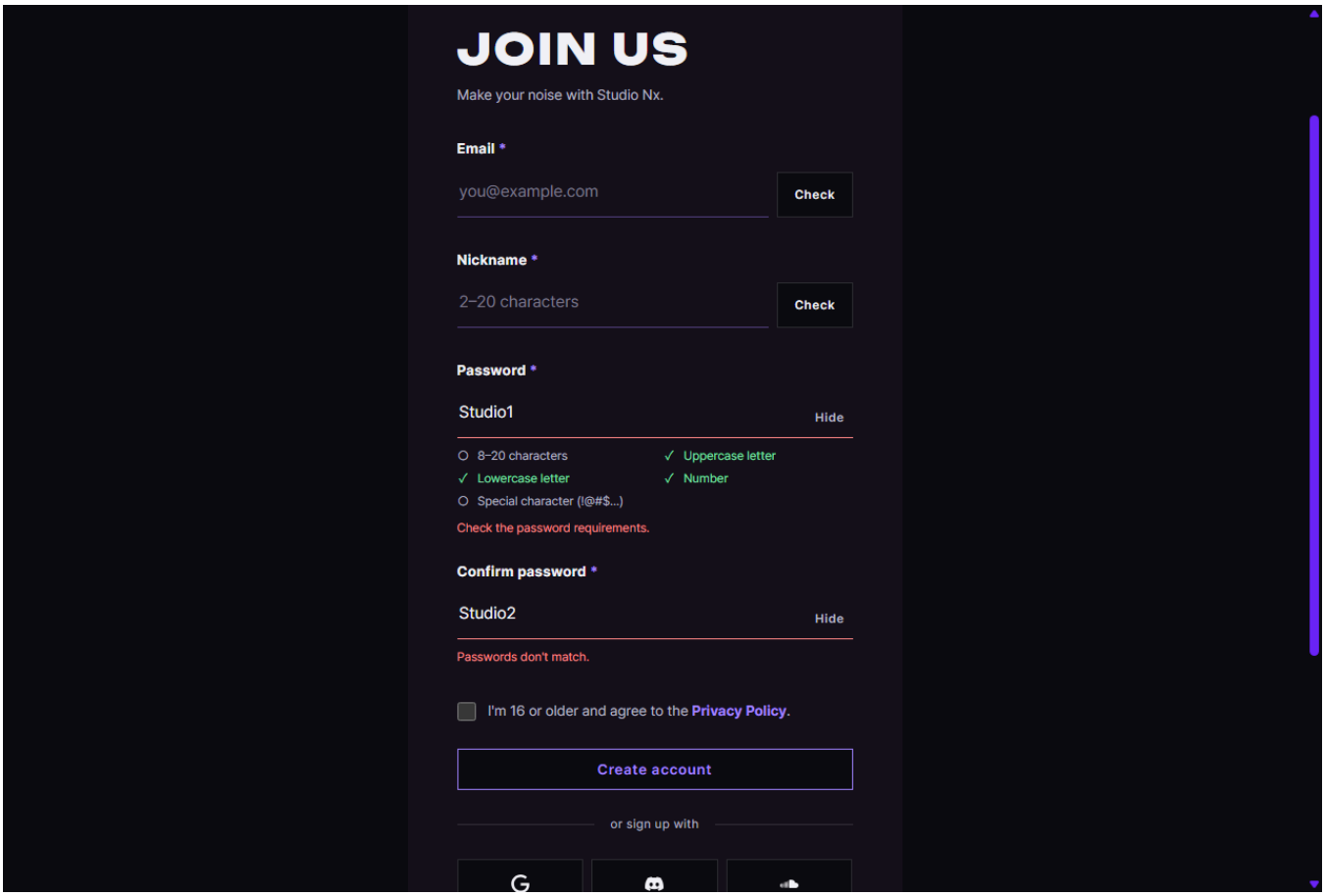
처음 돌렸을 때는 14/15였습니다. 위 §5의 빈틈(`data-checked`가 남음)을 고친 뒤 15개 모두 통과했습니다.

JS 16 — `initPasswordRules()`

항목	내용
파일	`studionx/frontend/static/js/pages/join.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-102: 비밀번호 조건을 입력할 때마다 ✓로 표시)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	15 `initDuplicateChecks()` · 17 `initJoinForm()`

1. 하는 일

join.html에서 비밀번호를 칠 때마다 아래 **조건 목록의 ○를 ✓(초록)**로 바꾸고, 틀린 점을 칸 아래에 알려 줍니다. 비밀번호 확인 칸의 값이 다르면 그것도 알려 줍니다.



입력	조건 목록	칸 아래
`a`	소문자만 ✓	(없음)
`Studio1` (입력 중)	대문자, 소문자, 숫자는 ✓이고, 8–20자와 기호는 ○	(없음: 입력 중에는 재촉하지 않습니다)
`Studio1`을 치고 칸을 벗어남	같음	빨강 "Check the password requirements."
`Studio1!`	모두 ✓	(빨간 글이 사라집니다)
`Studio1!한`, `Studio 1!`		**바로** 빨강 "Use only English letters, numbers and symbols."

입력	조건 목록	칸 아래
확인 칸에 `Studio2` (비밀번호 길이만큼 침)		빨강 "Passwords don't match."
확인 칸을 맞게 고침		**바로** 사라집니다

조건 목록은 화면 읽기 프로그램용 글자도 함께 바뀝니다 ("Uppercase letter, met" / "..., not met").

2. 언제 쓰나

join.html이 열릴 때 `join.js`가 한 번 부릅니다.

규칙	위치	같은 규칙을 쓰는 곳
`PASSWORD_RULES`(5개), `PASSWORD_ALLOWED`	`join.js` 위쪽(템플릿에 이미 있었음)에 있다가 **2026-10-08에 `core/password.js`로 옮겼습니다** (회원 정보 화면 JS 25와 함께 쓰고, `join.js`는 import합니다)	서버의 백엔드 14 `check_password_rules`입니다. 기호 모음이 같다는 것을 시험으로 확인했습니다

화면의 ✓는 **안내**입니다. 가입 버튼을 누르면 17번 `initJoinForm`이 같은 규칙으로 한 번 더 막고, 서버도 다시 검사합니다.

3. 코드

import (`join.js` 맨 위: 15번과 같고, 이번에 추가한 것은 없습니다)

```
import { initLoginForm, initPasswordToggles, initOAuthPlaceholders, setFieldState } from '../core/auth-ui.js';
…
```

이 함수는 **`setFieldState`**(09)를 쓰고, 규칙은 같은 파일의 `PASSWORD_RULES`와 `PASSWORD_ALLOWED`를 씁니다.

규칙 (템플릿에 있던 것)

```
export const PASSWORD_RULES = {
  length: (pw) => pw.length >= 8 && pw.length <= 20,
  upper: (pw) => /[A-Z]/.test(pw),
  lower: (pw) => /[a-z]/.test(pw),
  number: (pw) => /[0-9]/.test(pw),
  special: (pw) => /[!-/:-@[-`{-~]/.test(pw),
};
export const PASSWORD_ALLOWED = /^[!-~]*$/;
```

함수

```
export function initPasswordRules() {
  const password = document.getElementById('join-password');
  const confirm = document.getElementById('join-password2');
  const passwordField = document.getElementById('join-password-field');
  const confirmField = document.getElementById('join-password2-field');
  const items = document.querySelectorAll('#join-password-rules li[data-rule]');
  if (!password || !confirm) return;
```

```
  const allMet = () => {
    for (const name of Object.keys(PASSWORD_RULES)) {
      if (!PASSWORD_RULES[name](password.value)) return false;
    }
    return true;
  };

  const showRules = () => {
    for (const item of items) {
      const rule = PASSWORD_RULES[item.dataset.rule];
      const met = rule ? rule(password.value) : false;
      item.classList.toggle('is-met', met);
      const state = item.querySelector('[data-rule-state]');
      if (state) state.textContent = met ? ', met' : ', not met';
    }
  };

  const checkConfirm = (force) => {
    if (confirm.value === '') {
      setFieldState(confirmField, null);
    } else if (confirm.value === password.value) {
      setFieldState(confirmField, null);
    } else if (force || confirm.value.length >= password.value.length) {
      setFieldState(confirmField, 'error', "Passwords don't match.");
    }
  };

  password.addEventListener('input', () => {
    showRules();
    if (!PASSWORD_ALLOWED.test(password.value)) {
      setFieldState(passwordField, 'error', 'Use only English letters, numbers and symbols.');
    } else {
      setFieldState(passwordField, null);
    }
    if (confirm.value !== '') checkConfirm(true); // 확인 칸을 이미 쳤으면 바로 다시 비교
  });
  password.addEventListener('blur', () => {
    if (password.value !== '' && PASSWORD_ALLOWED.test(password.value) && !allMet()) {
      setFieldState(passwordField, 'error', 'Check the password requirements.');
    }
  });
  confirm.addEventListener('input', () => checkConfirm(false));
  confirm.addEventListener('blur', () => checkConfirm(true));
  showRules();
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const allMet = () => Object.values(PASSWORD_RULES).every((rule) => rule(password.value));
…
item.querySelector('[data-rule-state]')?.replaceChildren(met ? ', met' : ', not met');
…
const rule = PASSWORD_RULES[item.dataset.rule];
const met = rule?.(password.value) ?? false;
```

최신 문법	실제 코드	뜻
`Object.values(…).every(fn)`	`for … of Object.keys(…)`로 돌다가 하나라도 틀리면 `return false`	모두 참인지 확인합니다
`rule?.(password.value)`	`rule ? rule(password.value) : …`	함수가 있을 때만 부릅니다
`… ?? false`	`: false` (삼항 연산자의 뒷부분)	없으면 `false`입니다
`el?.replaceChildren(글)`	`if (state) state.textContent = 글`	있을 때만 글자를 바꿉니다

문법 풀이

import 문법	뜻
`export const PASSWORD_RULES`	17번 `initJoinForm`도 같은 규칙을 씁니다 (같은 파일)

식	뜻
`/[A-Z]/.test(pw)`	정규식입니다. `pw` 안에 대문자가 하나라도 있으면 `true`입니다
`/^[!-~]*$/`	처음(`^`)부터 끝(`$`)까지 모든 글자가 `!`(33)~`~`(126) 범위에 있는지 확인합니다. 영문, 숫자, 키보드 기호가 이 범위에 들고, 한글과 공백은 범위 밖입니다
`item.classList.toggle('is-met', met)`	`met`이 참이면 클래스를 붙이고, 거짓이면 뗍니다 (2015년 무렵부터 있던 기본 기능입니다)
`PASSWORD_RULES[item.dataset.rule]`	`<li data-rule="upper">`이면 `PASSWORD_RULES.upper` 함수를 가리킵니다
`'input'`, `'blur'`	글자가 바뀔 때마다, 그리고 칸에서 커서가 나갈 때 일어나는 이벤트입니다
`checkConfirm(force)`	`force`가 참이면 길이와 상관없이 "다름"을 보여 줍니다

4. 흐름

비밀번호를 칠 때: 조건 목록의 ✓/○를 바꿉니다. 허용되지 않는 글자가 있으면 바로 빨간 글을 보여 주고, 없으면 빨간 글을 지웁니다. 확인 칸에 이미 값이 있으면 다시 비교합니다. **비밀번호 칸을 벗어날 때:** 조건이 하나라도 맞지 않으면 "Check the password requirements."를 보여 줍니다. **확인 칸을 칠 때:** 같으면 문구를 지우고, 비밀번호 길이만큼 쳤는데 다르면 "다름"을 보여 줍니다. **확인 칸을 벗어날 때:** 다르면 길이와 상관없이 "다름"을 보여 줍니다.

5. 왜 이렇게

선택	이유
조건 미달은 칸을 **벗어날 때** 알림 (템플릿 대로)	치는 중에 "조건이 맞지 않음"을 계속 보여 주면 재촉하는 것처럼 느껴집니다. 조건 목록의 ✓가 이미 진행 상황을 보여 줍니다
허용되지 않는 글자는 **바로** 알림	한글 입력 상태로 치는 실수를 빨리 알아챌 수 있습니다. 계속 치면 바로잡기가 더 어렵습니다
확인 칸은 비밀번호 길이만큼 쳤을 때부터 "다름"을 보여 줌	한 글자 칠 때마다 "다름"이 뜨면 안 됩니다. 다 친 뒤에야 비교가 의미 있습니다
맞으면 바로 지움	고친 순간 확인받는 느낌을 줍니다
비밀번호를 고치면 확인 칸도 다시 비교	시험에서 찾은 경우입니다. 확인 칸까지 맞게 친 뒤 비밀번호 끝에 한 글자를 더하면, 확인 칸이 "맞음"인 채로 남아 있었습니다

선택	이유
✓는 CSS가 그림 (`li.is-met::before`)	JS는 클래스만 바꿉니다 (09번과 같은 원칙)
화면 읽기용 글자(`, met`)도 바꿈	색과 기호(✓)만으로는 화면을 보지 못하는 사람이 알 수 없습니다
규칙은 서버와 같게 맞춤	화면에서는 통과했는데 서버가 막는 일이 없게 합니다. 기호 모음이 같은지는 백엔드 14번 시험으로 확인했습니다

6. 시험 (실제 join.html을 헤드리스 Chrome으로 열고 그 안에서 실행, 17개)

시험	확인한 것
처음	모두 ○입니다
`a`, `Abcdef1`	소문자만 ✓ / 대문자, 소문자, 숫자 ✓이고, 화면 읽기용 글자가 `, met`과 `, not met`으로 바뀝니다
입력 중 / 칸을 벗어남	입력 중에는 빨간 글이 없고, 벗어나면 "Check the password requirements."가 나옵니다
`Abcdef1!`	모두 ✓이고, 빨간 글이 사라지며, 벗어나도 오류가 없습니다
한글, 공백	바로 "Use only English letters, numbers and symbols."가 나옵니다
24자	8–20자가 ○입니다
확인 칸	짧을 때는 말하지 않고, 비밀번호 길이만큼 쳤는데 다르면 "다름", 같아지면 지움, 짧은 채로 벗어나면 "다름", 비밀번호를 고치면 다시 "다름", 비우면 지웁니다

처음 돌렸을 때 두 가지가 실패했습니다.

- **시험 쪽 문제:** 페이지 스크립트가 끝나기 전에 시험이 시작되었습니다. 시험 도구가 2.5초 기다리도록 고쳤습니다.
- **실제 빈틈:** 비밀번호를 고쳐도 확인 칸을 다시 비교하지 않았습니다. §5에 적은 대로 고쳤습니다.

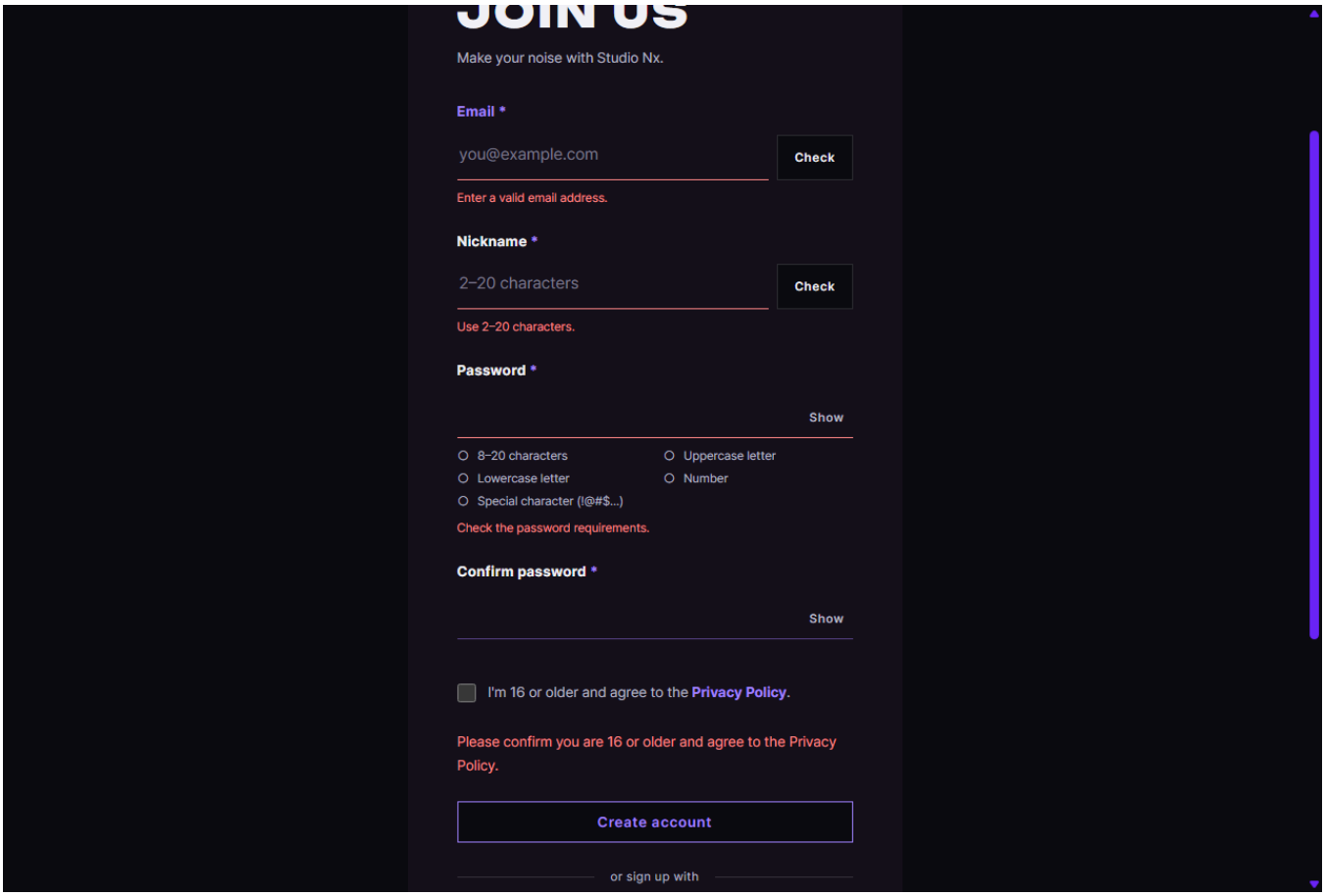
고친 뒤 17개 모두 통과했습니다. 앞의 09, 10, 11, 15번 시험도 다시 돌려서 모두 통과했습니다.

JS 17 — `initJoinForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/join.js`
작업	P1-07 가입 · 로그인 JS (PRD 01 FR-101, FR-102, 처리방침 동의; P1-07의 마지막 함수)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	16 `initPasswordRules()` · (P1-07 끝)

1. 하는 일

join.html의 **Create account**를 누르면 모든 칸을 검사하고, 맞으면 서버에 가입한 뒤 **로그인 모달이 열린 홈**으로 보냅니다. **이 함수 덕분에 사람이 화면에서 직접 가입할 수 있게 되었습니다.**



경우	화면
빈 칸, 형식 틀림	그 칸 아래에 빨간 문구가 나오고, 커서는 **첫 번째로 틀린 칸**에 둡니다. 서버에 보내지 않습니다
중복 확인 전에 누름 (입력 후 0.5초 안)	"Click Check to confirm this email is available."
비밀번호 조건, 확인 칸	"Check the password requirements.", "Passwords don't match."
동의를 체크하지 않음	버튼 위에 빨간 글씨로 "Please confirm you are 16 or older and agree to the Privacy Policy."가 나옵니다

경우	화면
모두 맞음	버튼이 잠깐 꺼지고, **`home.html#login`**으로 이동해서 로그인 모달이 자동으로 열립니다 (14)
확인한 뒤 다른 사람이 먼저 가입함 (서버 409)	그 칸에 "This email is already in use."가 나오고, 커서를 그 칸에 두며, 버튼을 다시 켭니다

함께 바꾼 한 줄: 가입 화면의 로그인 모달(이미 가입한 사람용)에서 로그인에 성공하면 홈으로 가게 했습니다.

```
initLoginForm({ onSuccess: (next) => { location.href = next || 'home.html'; } });
```

2. 언제 쓰나

join.html이 열릴 때 `join.js`가 한 번 부릅니다. 앞의 함수들이 만든 결과를 모아서 최종 판단을 합니다.

검사	근거	만든 곳
이메일, 닉네임 형식	`CHECKS[kind].isValid`	15
이메일, 닉네임 중복 확인 통과	칸의 `data-checked`가 지금 값과 같은지	15
비밀번호 조건, 허용 문자	`PASSWORD_RULES`, `PASSWORD_ALLOWED`	16
확인 칸	비밀번호와 같은지	16
동의	`#join-agree` 체크	join.html
서버	`POST /api/v1/auth/signup`	백엔드 21, 22

가입 한 번의 전체 흐름 (P1-04, 05, 07)

순서	화면	일어나는 일
①	join.html	이메일과 닉네임을 치면 0.5초 뒤 ✓가 나오고(15), 비밀번호도 ✓가 되며(16), 동의를 체크합니다
②	Create account	**이 함수**가 `POST /auth/signup`을 보내고 201을 받습니다
③	home.html#login	로그인 모달이 자동으로 열립니다 (14)
④	로그인	헤더가 My Page, My Projects로 바뀝니다 (13)

3. 코드

import (`join.js` 맨 위: 15번과 같고, 이번에 추가한 것은 없습니다)

```
import { initLoginForm, initPasswordToggles, initOAuthPlaceholders, setFieldState } from '../core/auth-ui.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { api } from '../core/api.js';
```

이 함수는 **`api`**와 **`setFieldState`**를 쓰고, 같은 파일의 `CHECKS`(15), `PASSWORD_RULES`, `PASSWORD_ALLOWED`, `PRIVACY_VERSION`을 씁니다.

함수 (길어서 세 부분으로 나눠 설명합니다)

```
export function initJoinForm() {
  const form = document.getElementById('join-form');
  if (!form) return;
  const fields = { email: …, nickname: …, password: …, password2: … };   // 칸(.field) 4개
  const email = …, nickname = …, password = …, password2 = …, agree = …, error = …, submit = …;

  const showFormError = (message) => {
    error.textContent = message;
    error.hidden = false;
  };
  const AGREE_MESSAGE = 'Please confirm you are 16 or older and agree to the Privacy Policy.';

  // 동의를 체크하면 동의 안내를 바로 지움 (다른 안내는 그대로) — 사이트 시험(ai-note 39)에서 찾아 추가
  agree.addEventListener('change', () => {
    if (agree.checked && error.textContent === AGREE_MESSAGE) error.hidden = true;
  });
```

① **화면에서 먼저 검사:** 틀린 칸의 입력 요소 목록을 돌려줍니다.

```
  const findProblems = () => {
    const problems = [];
    for (const kind of ['email', 'nickname']) {
      const input = kind === 'email' ? email : nickname;
      const value = input.value.trim();
      if (!CHECKS[kind].isValid(input)) {
        setFieldState(fields[kind], 'error', CHECKS[kind].invalid);
        problems.push(input);
      } else if (fields[kind].dataset.checked !== value) {
        if (!fields[kind].classList.contains('is-error')) {
          setFieldState(fields[kind], 'error', `Click Check to confirm this ${kind} is available.`);
        }
        problems.push(input);
      }
    }
    let rulesMet = PASSWORD_ALLOWED.test(password.value);
    for (const name of Object.keys(PASSWORD_RULES)) {
      if (!PASSWORD_RULES[name](password.value)) rulesMet = false;
    }
    if (!rulesMet) {
      setFieldState(fields.password, 'error', 'Check the password requirements.');
      problems.push(password);
    }
    if (password2.value !== password.value) {
      setFieldState(fields.password2, 'error', "Passwords don't match.");
      problems.push(password2);
    }
    if (!agree.checked) {
      showFormError(AGREE_MESSAGE);
      problems.push(agree);
    }
    return problems;
  };
```

② **서버가 막은 이유를 그 칸 아래에 표시** (409, 422)

```
  const showServerError = (err) => {
    const details = err.details || {};
    if (err.code === 'EMAIL_TAKEN' || details.email) {
      delete fields.email.dataset.checked;
      setFieldState(fields.email, 'error', err.code === 'VALIDATION_ERROR' ? CHECKS.email.invalid : err.
message);
      email.focus();
    } else if (err.code === 'NICKNAME_TAKEN' || details.nickname) {
      delete fields.nickname.dataset.checked;
      setFieldState(fields.nickname, 'error', err.code === 'VALIDATION_ERROR' ? CHECKS.nickname.invalid
: err.message);
      nickname.focus();
    } else if (details.password) {
      setFieldState(fields.password, 'error', 'Check the password requirements.');
      password.focus();
    } else {
      showFormError(err.message || 'Something went wrong. Please try again.');
    }
  };
```

③ 보내기

```
  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    error.hidden = true;
    const problems = findProblems();
    if (problems.length > 0) {
      problems[0].focus();
      return;
    }
    submit.disabled = true;
    try {
      await api.post('/auth/signup', {
        email: email.value.trim(), nickname: nickname.value.trim(), password: password.value,
        privacy_agreed: true, privacy_version: PRIVACY_VERSION,
      }, { auth: false });
      location.href = 'home.html#login';
    } catch (err) {
      showServerError(err);
      submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const rulesMet = PASSWORD_ALLOWED.test(password.value)
  && Object.values(PASSWORD_RULES).every((rule) => rule(password.value));
…
const { code, message, details = {} } = err;
…
problems.at(0)?.focus();
```

최신 문법	실제 코드	뜻
`Object.values(…).every(fn)`	`for … of`로 돌다가 하나라도 틀리면 `false`	모두 참인지 확인합니다
`const { code, details = {} } = err`	`err.code`, `const details = err.details || {}`	꺼내면서, 없으면 기본값을 씁니다
`problems.at(0)?.focus()`	`if (problems.length > 0) problems[0].focus()`	첫 번째가 있으면 실행합니다

문법 풀이

import 문법	뜻
(추가 없음)	15번에서 넣은 `api`와 처음부터 있던 `setFieldState`를 씁니다

식	뜻
`<form … novalidate>` (join.html)	브라우저의 기본 검사 말풍선을 끕니다. 대신 이 함수가 칸 아래 문구로 직접 보여 줍니다 (13번의 로그인 폼은 브라우저 검사를 씁니다)
`const findProblems = () => { … return problems; }`	안쪽 함수입니다. 틀린 칸들을 모아서 돌려줍니다
`problems.push(input)`	목록 끝에 넣습니다
`problems[0].focus()`	화면 위쪽부터 검사했으므로, 목록의 첫 번째는 **가장 위에 있는 틀린 칸**입니다
`` `Click Check to confirm this ${kind} is available.` ``	`kind`가 `'email'`이면 "… this email is available."이 됩니다
`err.code`, `err.details`	`ApiError`가 서버 오류 JSON에서 꺼내 둔 값입니다
`location.href = 'home.html#login'`	홈으로 이동하고, 14번이 로그인 모달을 엽니다

4. 흐름

1. 폼을 보낼 때 폼 전체의 오류 글을 숨깁니다.
2. 화면에서 모든 칸을 검사합니다. 틀린 칸이 있으면 문구를 붙이고 첫 칸에 커서를 둔 뒤 끝냅니다.
3. 버튼을 끄고 서버에 가입을 요청합니다.
4. 성공하면 `home.html#login`으로 이동합니다. 실패하면 그 칸(또는 폼 아래)에 문구를 보여 주고 버튼을 다시 켭니다.

5. 왜 이렇게

선택	이유
모든 칸을 검사한 뒤 첫 칸에 커서를 둠	하나씩 고치고 다시 누르게 하지 않고, 틀린 곳을 한 번에 다 보여 줍니다
중복 확인을 통과한 값만 보냄 (`data-checked`)	템플릿의 요구 사항입니다. 보내기 전에 "이미 사용 중"인 것을 알면 서버까지 갔다 올 필요가 없습니다. 보통은 입력 후 0.5초 자동 확인(15)이 끝나 있으므로 따로 누를 일이 없습니다
이미 빨간 문구(예: 사용 중)가 있으면 그대로 둠	"Click Check…"로 덮으면 왜 안 되는지가 사라집니다
서버의 409, 422를 그 칸에 표시	확인과 가입 사이에 다른 사람이 먼저 가입하는 경우(백엔드 21의 동시 가입)에도, 사용자는 어느 칸을 고쳐야 할지 압니다

선택	이유
가입 후 자동 로그인을 하지 않고 로그인 모달을 엶 (템플릿 선택지 중 하나)	가입과 로그인을 나눈 서버 설계(백엔드 22)에 맞춘 것입니다. 사용자가 "Keep me signed in"을 직접 고를 수 있습니다
성공하면 버튼을 다시 켜지 않음	곧 다른 페이지로 이동하므로, 그사이 또 누르는 것을 막습니다
동의를 체크하면 동의 안내를 바로 지움	사이트 시험(ai-note 39)에서 찾은 빈틈입니다. 체크했는데도 빨간 "Please confirm..."이 다음 전송 때까지 남아 있었습니다. 서버 오류 같은 다른 안내는 지우지 않습니다
가입 화면의 로그인 모달에서 로그인하면 홈으로 이동	가입 화면에서 로그인한 사람이 가입 화면에 남아 있을 이유가 없습니다. `next`가 있으면 그쪽으로 갑니다

6. 시험 (실제 join.html에서 실제 서버로, 10개)

시험	확인한 것
빈 채로 보냄	이메일, 닉네임, 비밀번호 문구와 동의 문구가 나오고, 가입 요청은 0번이며, 커서는 이메일 칸에 있습니다
다 맞게 치고 0.5초 안에 보냄	"Click Check to confirm this email/nickname is available."이 나오고, 요청은 0번입니다
자동 확인 뒤에 보냄	둘 다 ✓가 되고, 보내는 동안 버튼이 꺼지며, 요청을 1번 보낸 뒤 `/home.html#login`으로 이동하려 합니다 (시험에서는 이동만 막았습니다). 서버에서 그 이메일은 이제 `TAKEN`입니다
확인을 통과한 것처럼 꾸민 `demo@example.com`으로 보냄	서버가 409로 응답하면 "This email is already in use."가 나오고, 통과 표시를 지우며, 버튼을 다시 켜고, 커서를 이메일 칸에 둡니다
가입 화면의 로그인 모달로 로그인	`/home.html`로 이동하려 합니다
동의 없이 보낸 뒤 체크 (사이트 시험 뒤에 추가)	동의 안내가 바로 사라집니다

처음에는 9개 모두 통과했습니다. 사이트 시험에서 동의 안내가 남는 빈틈을 찾아 고친 뒤 시험 1개를 더해서 10개 모두 통과했습니다. 앞 함수들(09~16)의 시험도 다시 돌려서 모두 통과했습니다. 시험에서 만든 계정은 끝난 뒤 지웠습니다 (데모 계정만 남겼습니다).